EVENT BOOKING SYSTEM

Aaron Mathews Molayal	bs24dmu023	Backend & Cloud Architecture (Lead)
Tselmeg Od	bj25dsy009	Frontend Development, AI Integration, Deployment
Aisyah Ihsani Bangsawan Putri	bj25dsy004	QA Testing & Validation
Hayyan Mohammed	bs24dsy055	Architecture Diagram
Chalisa Nabila Afrizal	bj25dsy007	PPT Design & Documentation

Index Page

Section	Topic	~Page no	Deliverables
A	Overview, Problem and Goal	2	
B	App System Features and Pages	3	
C	Frontend Structure	4	
D	Core Backend Structure	5	
E	Database Structure	6	
F	Cloud Architecture Diagram	7	yes
G	Design Choice Justification	8	yes
H	Image Proof of Key Components	10	yes
I	Application Testing Proof	16	yes
J	Mandatory Testing Proof	23	yes
K	Step by Step Proof of Cloud Deployment	27	
L	AI Integration and System Enhancements	45	

Event Booking Platform

A. Overview, Problem & Goal

1. Overview

- Cloud-based web application for browsing events and reserving seats
- Admin interface for creating events, managing bookings, and monitoring demand
- Deployed on AWS to demonstrate:
  - Scalability
  - Security
  - High availability
  - Reliable database management

2. Business Problem

- Event organizers rely on manual systems:
  - Phone calls
  - Messaging apps
  - Spreadsheets
- Issues caused:
  - Booking errors
  - No real-time seat visibility
  - Poor handling of high demand
- Lack of centralized system leads to:
  - Difficulty tracking attendance
  - Inefficient capacity management
  - Slow response to customer requests

3. Solution Goal

- Build a **secure, scalable, and highly available** event booking platform on AWS
- System capabilities:
  - Simple and reliable user interface for booking events
  - Admin tools for managing events and bookings
  - Handle traffic spikes using load balancing and auto scaling
  - Ensure reliability using managed cloud services (e.g., RDS Multi-AZ)
  - Maintain security through layered network design and access control

B. System Features and Pages

1. User Features

- Browse available events
- View event details:
  - Date
  - Location
  - Capacity
- Reserve seats for an event
- Receive booking confirmation:
  - Booking success / status page
- AI Assistance:
  - AI-powered event improvement (suggests better descriptions/details)
  - AI-based search/chat assistance for finding events

2. Admin Features

Administrators can:

- {email: 'admin@gmail.com', password: '12345'}
- **Event Management**
  - Create new events
  - Update event details
  - Delete events
- **Booking Management**
  - View all bookings
  - Update booking status
- **Monitoring**
  - View platform activity through dashboard

3. Bonus Features

Event Capacity Enforcement

- Each event has a maximum seat limit
- Backend:
  - Checks availability before confirming booking
  - Prevents overbooking
- Frontend:
  - Displays **"Sold Out"** when capacity is reached

Admin Analytics Dashboard

- Displays key metrics:
  - Total events
  - Total bookings
  - Most popular events
- Helps admins:
  - Track demand

- Make better decisions

AI-Based Features

- AI-powered event improvement:
  - Enhances event descriptions using AI
- AI-based search/chat assistance:
  - Helps users find events through natural language queries

4. Frontend Pages

User Pages

- Home / Landing Page
- Events List Page
- Event Details Page
- Booking Page
- Booking Success / Confirmation Page

Admin Pages

- Admin Login Page
- Admin Dashboard
- Manage Events Page
- Manage Bookings Page
- Admin Logout

5. Security

- Admin Login password is encrypted in database(admins table) using bcrypt

---

C. Frontend Structure

User Routes

/	Home page
/events	View all available events
/events/:id	View details of a specific event
/booking-success	Booking confirmation page shown after a successful booking

Admin Routes

/admin/login	Admin login page
/admin	Admin dashboard
/admin/bookings	Manage bookings
/admin/events	Manage events

D. Core Backend Structure

Request Flow

- Request → Middleware → Routes → Controllers → Models → Database → Response

Explanation

- **Middleware** handles request processing such as validation, authentication, and error checking
- **Routes** define which endpoint is being accessed
- **Controllers** process request logic and decide what action to perform
- **Models** interact with the database
- **Database** stores and retrieves application data
- **Response** sends the final output back to the frontend
- AI service integrated via backend API
- Uses external OpenAI model to enhance event data and user interaction

API Endpoints

Public Endpoints

GET /events	Retrieve all available events
GET /events/:id	Retrieve details of a specific event
POST /bookings	Create a new booking
POST /auth/login	admin login authentication endpoint

Admin Endpoints

POST /admin/events	Create a new event
PUT /admin/events/:id	Update an existing event
DELETE /admin/events/:id	Delete an event
GET /admin/bookingsWithTitles	Retrieve all bookings with event titles
PATCH /admin/bookings/:id/status	Update booking status

Infrastructure Endpoint

GET /health	Health check endpoint used by the load balancer to verify backend availability

E. Database Schema

The system uses a **relational database** to store event, booking, and admin data.

1. Events Table

Field	Description
id	Unique event identifier
title	Event name
description	Event description
date	Event date and time
location	Event location
total_capacity	Maximum number of seats
available_capacity	Remaining seats available
category	Event category
created_at	Record creation timestamp

---

2. Bookings Table

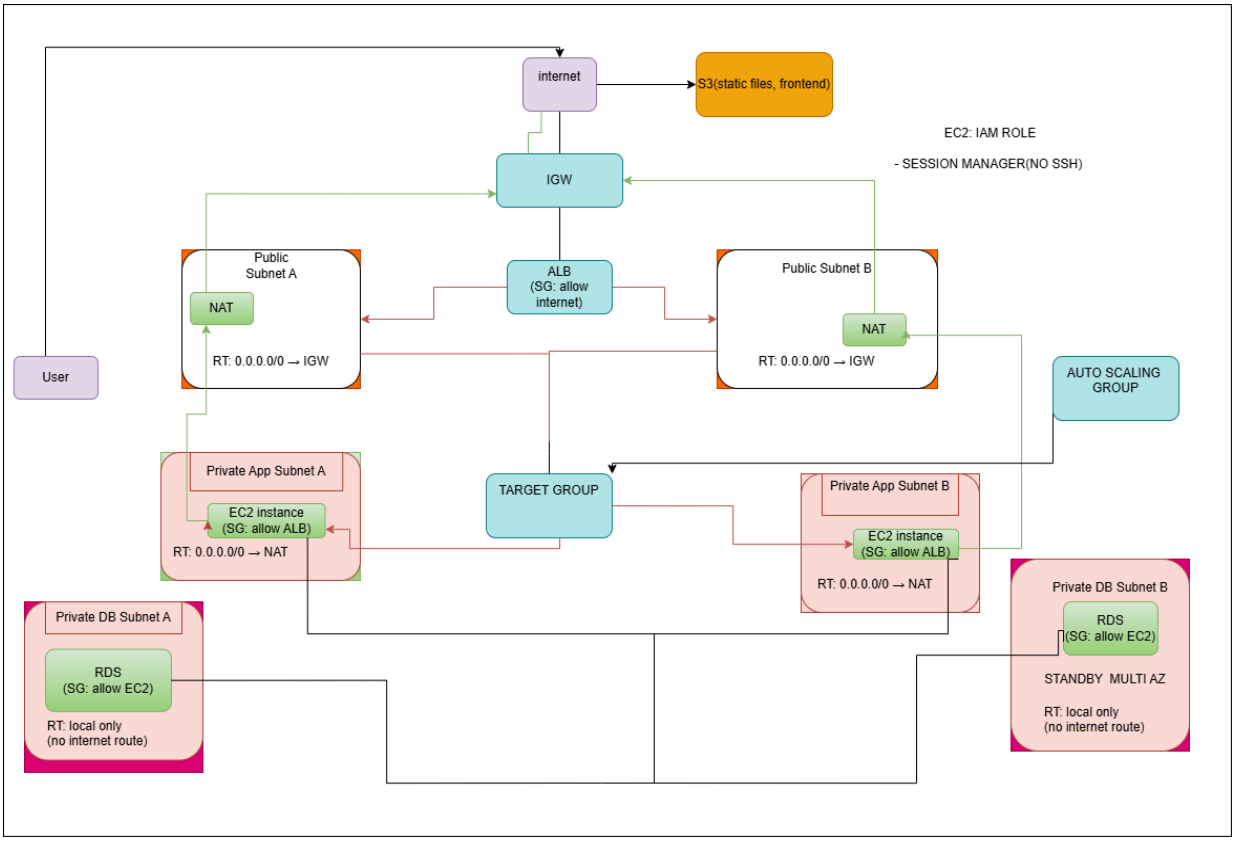
Field	Description
id	Unique booking identifier
event_id	Associated event
customer_name	Name of the user
customer_email	User contact information
seats_booked	Number of seats reserved
booking_status	Booking state (confirmed, pending, cancelled)
created_at	Booking timestamp

---

3. Admins Table

Field	Description
id	1
name	Admin name
email	Admin email
password_hash	Hashed password stored using bcrypt

F. Cloud Architecture Diagram



Traffic Flow

User → Internet → ALB → EC2 → RDS

Frontend Flow

User → Internet → S3

Response Flow

RDS → EC2 → ALB → Internet → User

Private Internet Access

EC2 → NAT Gateway → Internet

Auto Scaling Flow

ASG → EC2 instances

Health Check Flow

ALB → EC2 (/health)

Security Groups

- ALB SG: Allows traffic from internet
- EC2 SG: Allows traffic only from ALB
- RDS SG: Allows traffic only from EC2

G. Design Choice Justification

The following components were selected to ensure a secure, scalable, and highly available cloud architecture.

1. VPC

What it is: A logically isolated virtual network in AWS.

Role in our system: Hosts all components (ALB, EC2, RDS) in a controlled network environment.

Why we chose it: Provides isolation and allows us to design secure public and private subnet architecture as required.

2. Public and Private Subnets

What it is: Subdivisions of a VPC with different access levels.

Role in our system: Public subnets host ALB and NAT Gateway, while private subnets host EC2 and RDS.

Why we chose it: Ensures backend and database are not directly exposed to the internet, improving security.

3. Application Load Balancer (ALB)

What it is: A service that distributes incoming traffic across multiple targets.

Role in our system: Routes user requests to healthy EC2 backend instances.

Why we chose it: Improves availability and ensures traffic is only sent to healthy instances using health checks across multiple AZs

4. Auto Scaling Group (ASG)

What it is: Automatically manages the number of EC2 instances.

Role in our system: Maintains at least 2 instances and replaces failed ones.

Why we chose it: Ensures high availability and satisfies requirement for fault tolerance.

5. EC2

What it is: Virtual servers in AWS.

Role in our system: Runs backend application logic.

Why we chose it: Provides scalable compute to run backend services and integrate with ASG for high availability

6. RDS (Multi-AZ)

What it is: Managed relational database service with automatic failover.

Role in our system: Stores application data securely in private subnets.

Why we chose it: Multi-AZ ensures automatic failover and high availability of the database

7. NAT Gateway

What it is: Allows private instances to access the internet.

Role in our system: Allows EC2 instances in private subnets to access the internet for updates without being publicly accessible

Why we chose it: Maintains security while allowing required outbound access.

8. Internet Gateway (IGW)

What it is: Connects VPC to the internet.

Role in our system: Allows users to reach the ALB.

Why we chose it: Required for public-facing components.

9. Security Groups

What it is: Virtual firewalls controlling traffic.

Role in our system:

• ALB accepts traffic from internet
• EC2 accepts traffic only from ALB
• DB accepts traffic only from EC2

Why we chose it: Implements least-privilege security model.

10. IAM Role + Session Manager

What it is: IAM role grants permissions; Session Manager enables secure access.

Role in our system: Allows access to EC2 without SSH.

Why we chose it: Improves security by removing need for open SSH ports.

11. S3 (Frontend)

What it is: Object storage service.

Role in our system: Hosts static frontend files.

Why we chose it: Simple, scalable, and cost-effective for static hosting.

12. Route Tables

What it is: Rules that control where network traffic is routed within a VPC

Role in our system:

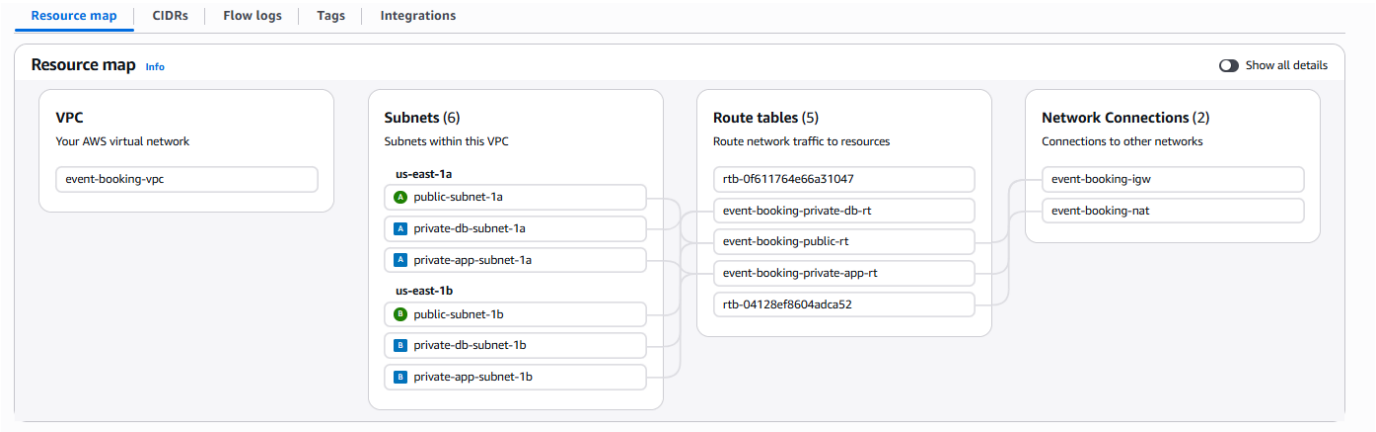
- Public subnets → route internet traffic to Internet Gateway
- Private app subnets → route outbound traffic to NAT Gateway
- Private DB subnets → no internet route (internal only)

Why we chose it: Ensures correct traffic flow while keeping backend

and database private and secure

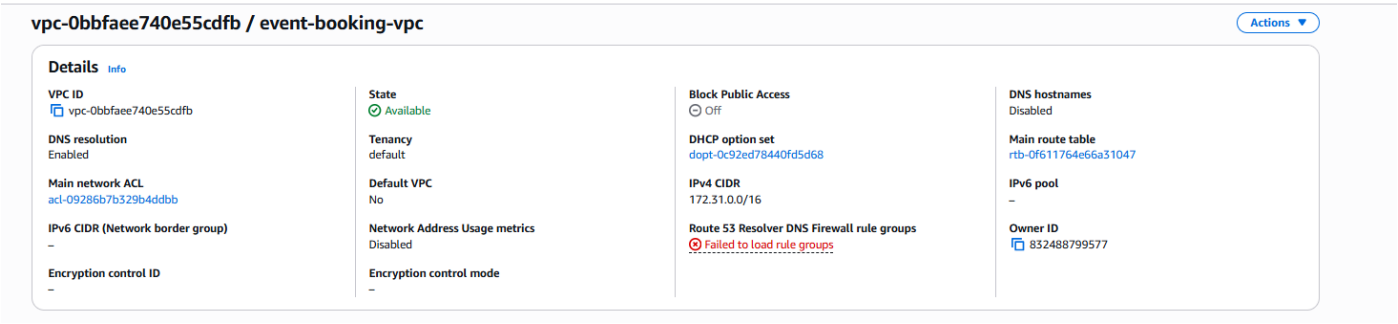
H. Image Proof of Key Components

• Resource map



• VPC

event-booking-vpc

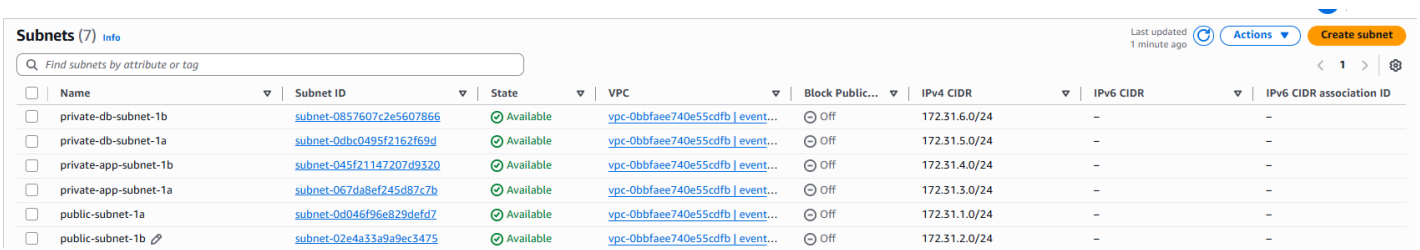


• Subnets

6 subnets [2 public + 2 private (for DB) + 2 private (for application)]

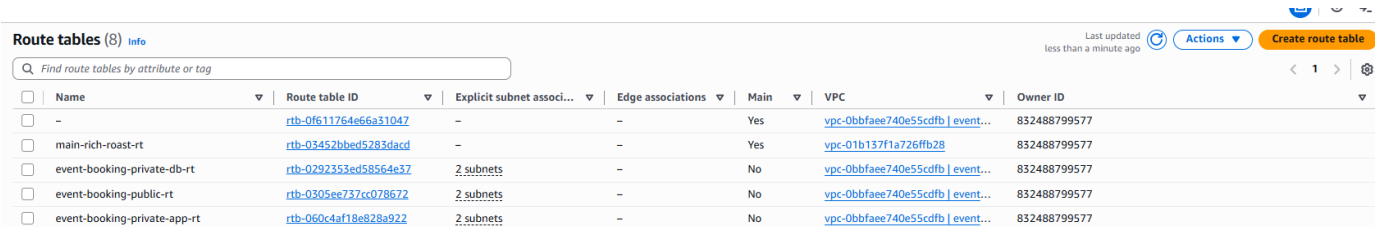
Subnets:

- public-subnet-1a
- public-subnet-1b
- private-subnet-app-1a
- private-subnet-app-1b
- private-subnet-db-1a
- private-subnet-db-1b



- **Route Tables**

  event-booking-public-rt
  event-booking-private-app-rt
  event-booking-private-db-rt



Route tables (8) Info

Last updated less than a minute ago | Actions | Create route table

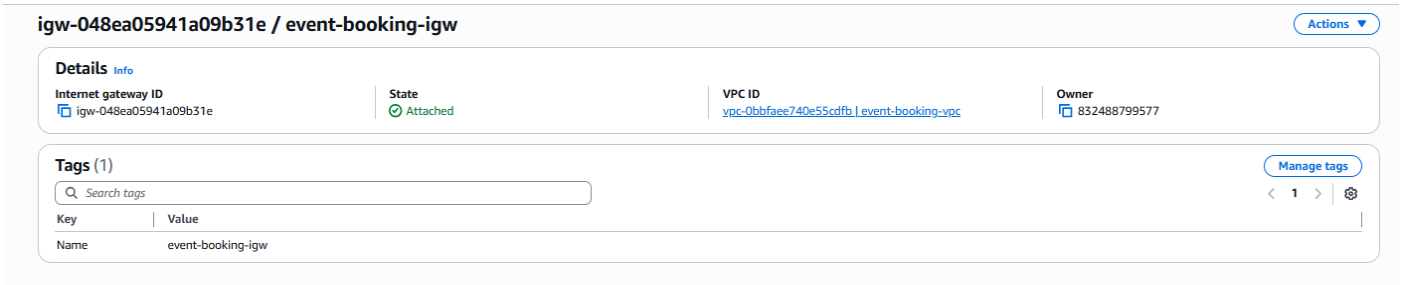
Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
–	rtb-0f611764e66a31047	–	–	Yes	vpc-0bbfaee740e55cdfb | event...	832488799577
main-rich-roast-rt	rtb-03452bbed5283dacd	–	–	Yes	vpc-01b137f1a726ffb28	832488799577
event-booking-private-db-rt	rtb-0292353ed58564e37	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577
event-booking-public-rt	rtb-0305ee737cc078672	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577
event-booking-private-app-rt	rtb-060c4af18e828a922	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577

- **IGW and NAT**

  event-booking-igw

e



igw-048ea05941a09b31e / event-booking-igw

Actions

Details Info

Internet gateway ID	State	VPC ID	Owner
igw-048ea05941a09b31e	Attached	vpc-0bbfaee740e55cdfb | event-booking-vpc	832488799577

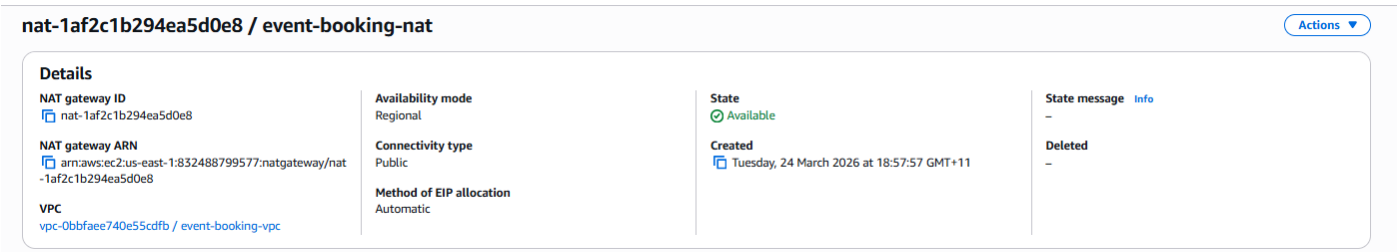
Tags (1)

Manage tags

Search tags

Key	Value
Name	event-booking-igw

event-booking-nat



nat-1af2c1b294ea5d0e8 / event-booking-nat

Actions

Details

NAT gateway ID
nat-1af2c1b294ea5d0e8

NAT gateway ARN
arn:aws:ec2:us-east-1:832488799577:natgateway/nat-1af2c1b294ea5d0e8

VPC
vpc-0bbfaee740e55cdfb / event-booking-vpc

Availability mode
Regional

Connectivity type
Public

Method of EIP allocation
Automatic

State
Available

Created
Tuesday, 24 March 2026 at 18:57:57 GMT+11

State message Info
–

Deleted
–

- **Security Groups**

  event-booking-alb-sg
  event-booking-app-sg
  event-booking-db -sg

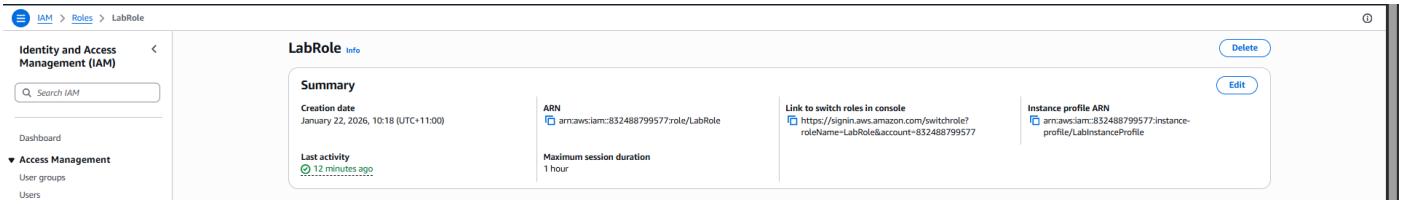
Security Groups (10)

Actions | Export security groups to CSV | Create security group

Find security groups by attribute or tag

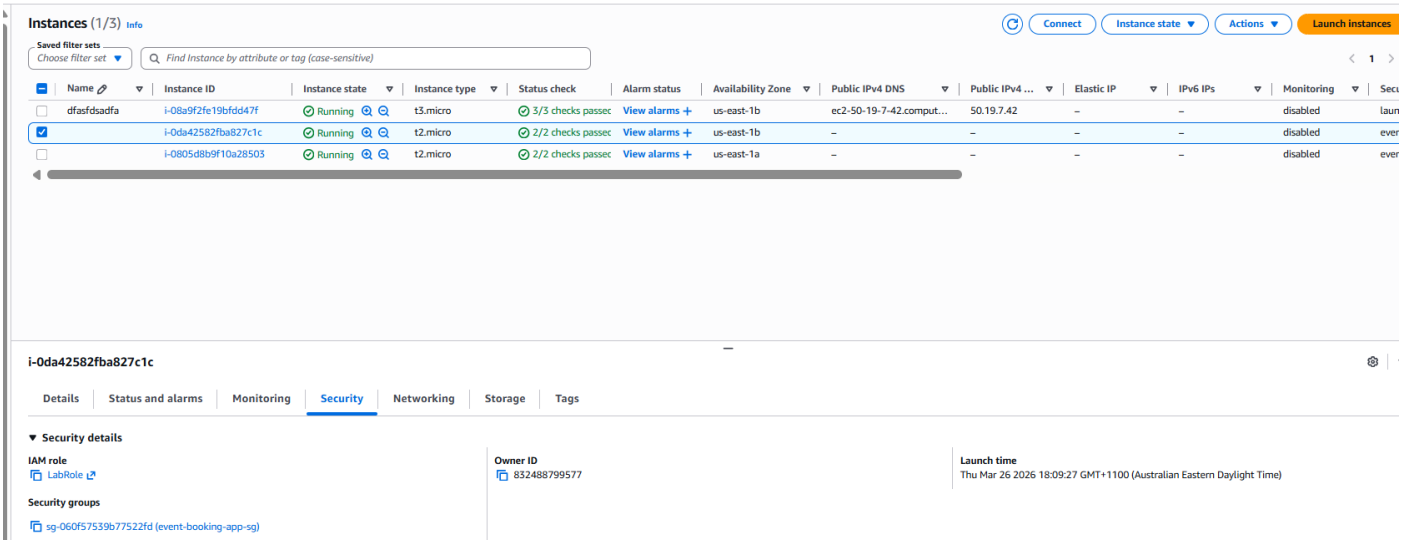
Security group ID	Security group name	VPC ID	Description	Owner	Inbound rules count	Outbound rules count
sg-007cb4e791950ef9a	event-booking-db-sg	vpc-0bbfaee740e55cdfb	traffic from application	832488799577	1 Permission entry	1 Permission entry
sg-060f57539b77522fd	event-booking-app-sg	vpc-0bbfaee740e55cdfb	traffic from alb	832488799577	1 Permission entry	1 Permission entry
sg-01051730089dbdba4	event-booking-alb-sg	vpc-0bbfaee740e55cdfb	traffic from internet (eb1)	832488799577	2 Permission entries	1 Permission entry

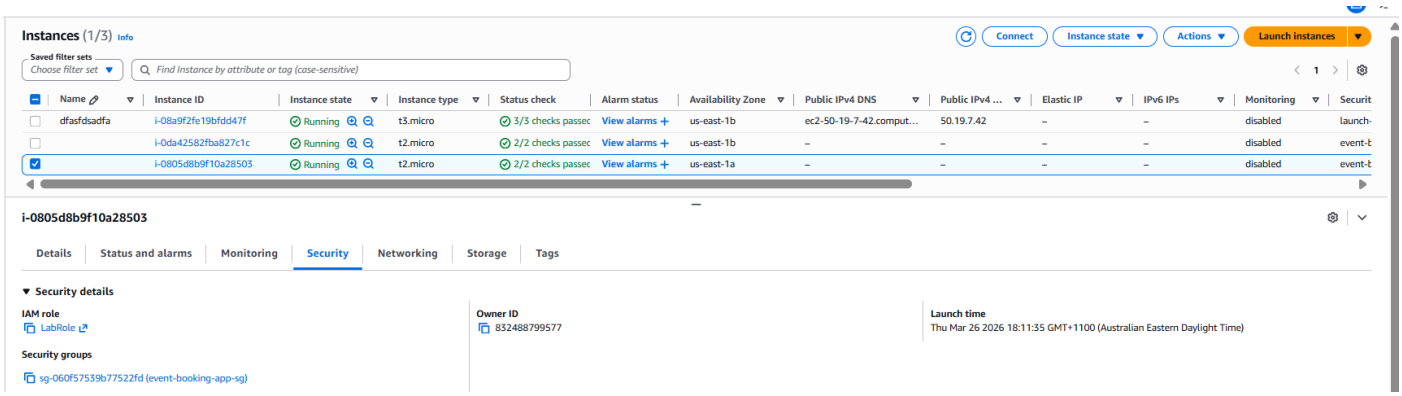
- **IAM Role Attached to EC2** (Due to account restrictions, LabRole/LabInstanceRole was used)



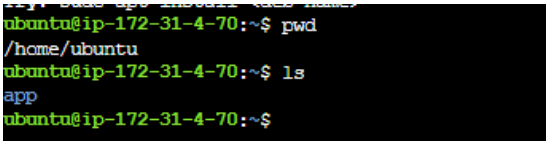
- **Session Manager Working**

1. Instances Proof:

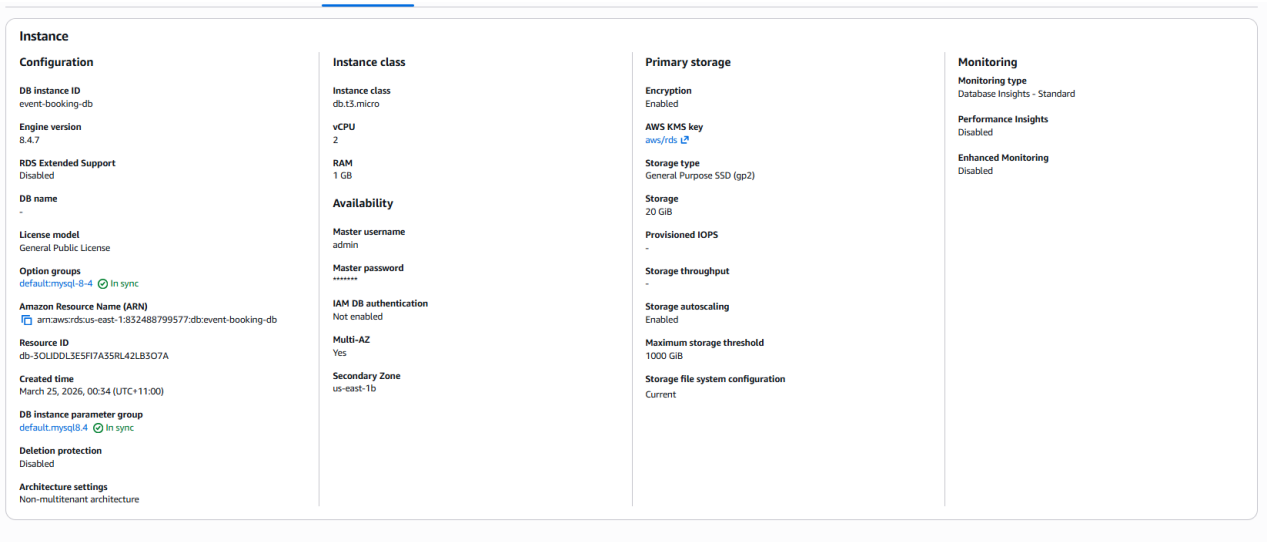




Session Manager Working Proof:



- **RDS Multi-AZ Config**



Instance

Configuration

DB instance ID
event-booking-db

Engine version
8.4.7

RDS Extended Support
Disabled

DB name
-

License model
General Public License

Option groups
default:mysql-8-4 In sync

Amazon Resource Name (ARN)
arn:aws:rds:us-east-1:832488799577:db:event-booking-db

Resource ID
db-3OLIDDL3E5FI7A35RL42LB3O7A

Created time
March 25, 2026, 00:34 (UTC+11:00)

DB instance parameter group
default.mysql8.4 In sync

Deletion protection
Disabled

Architecture settings
Non-multitenant architecture

Instance class

Instance class
db.t3.micro

vCPU
2

RAM
1 GB

Availability

Master username
admin

Master password
****

IAM DB authentication
Not enabled

Multi-AZ
Yes

Secondary Zone
us-east-1b

Primary storage

Encryption
Enabled

AWS KMS key
aws/rds

Storage type
General Purpose SSD (gp2)

Storage
20 GiB

Provisioned IOPS
-

Storage throughput
-

Storage autoscaling
Enabled

Maximum storage threshold
1000 GiB

Storage file system configuration
Current

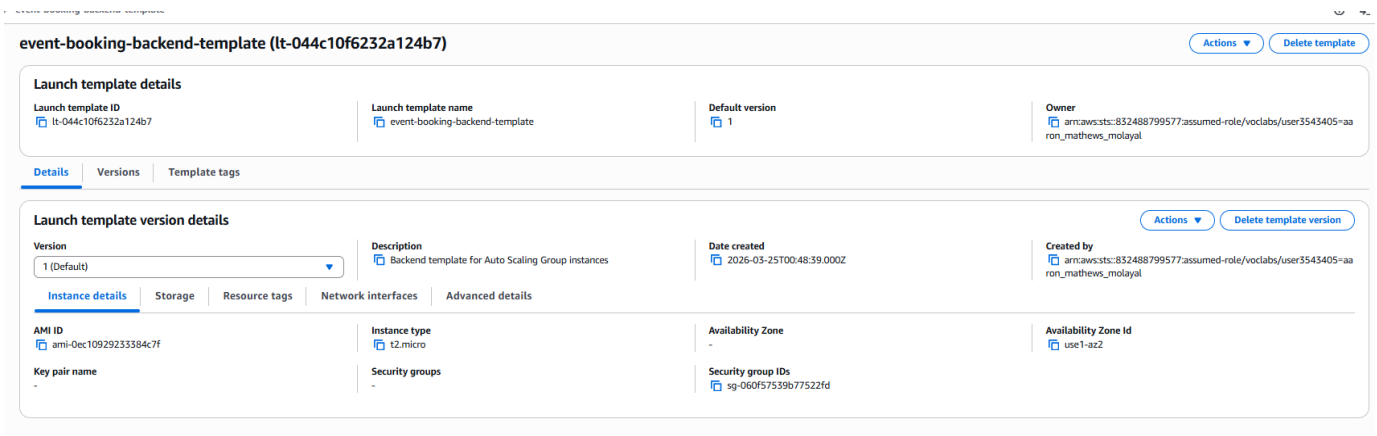
Monitoring

Monitoring type
Database Insights - Standard

Performance Insights
Disabled

Enhanced Monitoring
Disabled

- **Launch Template**



event-booking-backend-template (lt-044c10f6232a124b7)

Actions | Delete template

Launch template details

Launch template ID
lt-044c10f6232a124b7

Launch template name
event-booking-backend-template

Default version
1

Owner
arn:aws:sts::832488799577:assumed-role/voclabs/user3543405=aaron_mathews_molayal

Details | Versions | Template tags

Launch template version details

Actions | Delete template version

Version
1 (Default)

Description
Backend template for Auto Scaling Group instances

Date created
2026-03-25T00:48:39.000Z

Created by
arn:aws:sts::832488799577:assumed-role/voclabs/user3543405=aaron_mathews_molayal

Instance details | Storage | Resource tags | Network interfaces | Advanced details

AMI ID
ami-0ec10929233384c7f

Instance type
t2.micro

Availability Zone
-

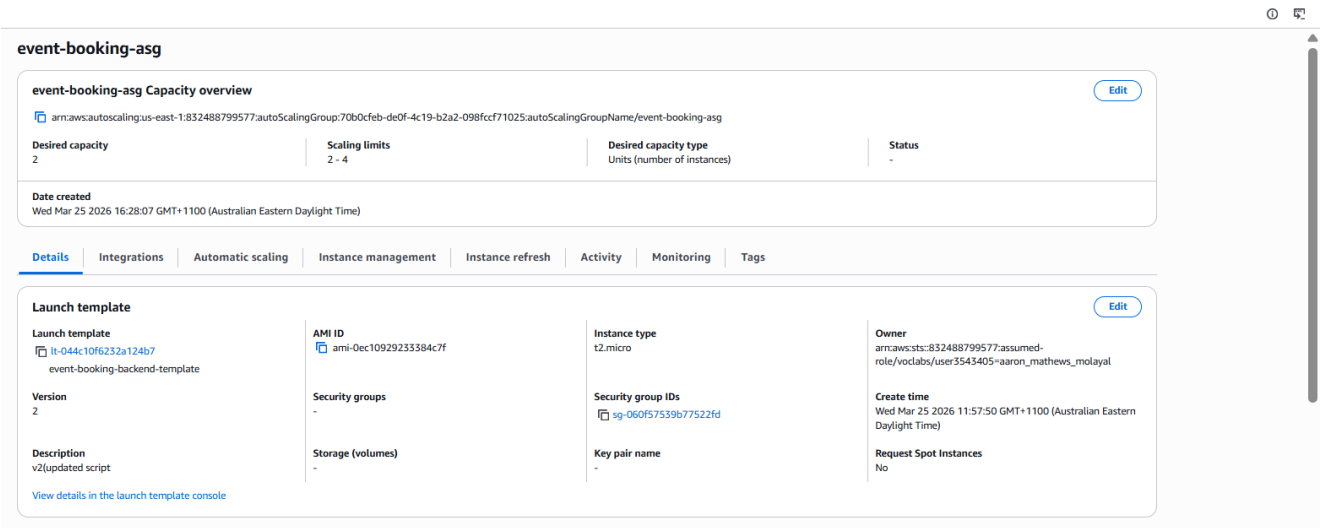
Availability Zone Id
use1-az2

Key pair name
-

Security groups
-

Security group IDs
sg-060f57539b77522fd

- **ASG proof**



event-booking-asg

event-booking-asg Capacity overview

Edit

arn:aws:autoscaling:us-east-1:832488799577:autoScalingGroup:70b0cfeb-de0f-4c19-b2a2-098fccf71025:autoScalingGroupName/event-booking-asg

Desired capacity
2

Scaling limits
2 - 4

Desired capacity type
Units (number of instances)

Status
-

Date created
Wed Mar 25 2026 16:28:07 GMT+1100 (Australian Eastern Daylight Time)

Details | Integrations | Automatic scaling | Instance management | Instance refresh | Activity | Monitoring | Tags

Launch template

Edit

Launch template
lt-044c10f6232a124b7
event-booking-backend-template

AMI ID
ami-0ec10929233384c7f

Instance type
t2.micro

Owner
arn:aws:sts::832488799577:assumed-role/voclabs/user3543405=aaron_mathews_molayal

Version
2

Security groups
-

Security group IDs
sg-060f57539b77522fd

Create time
Wed Mar 25 2026 11:57:50 GMT+1100 (Australian Eastern Daylight Time)

Description
v2(updated script

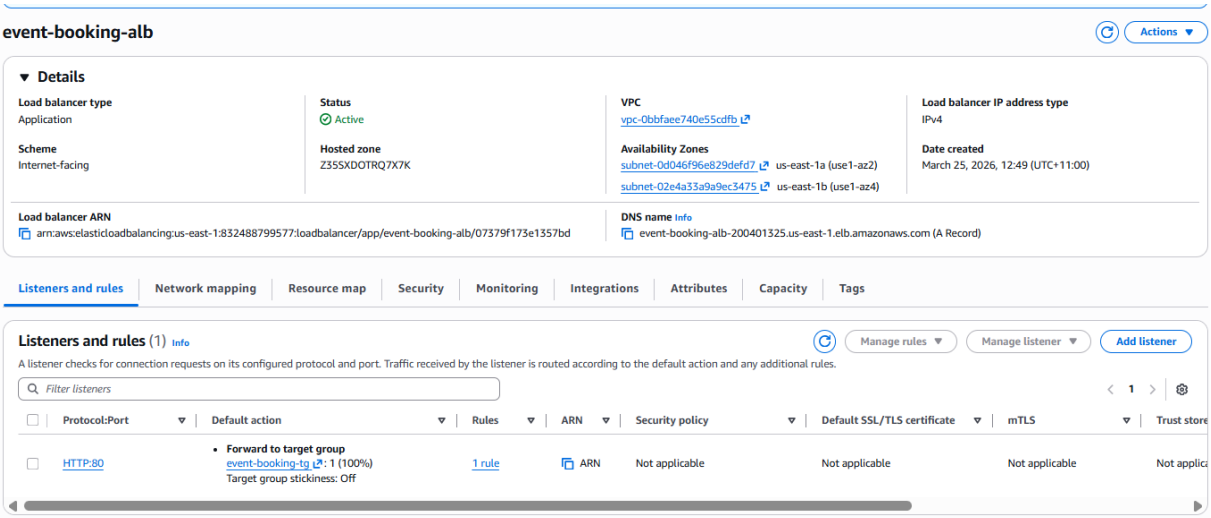
Storage (volumes)
-

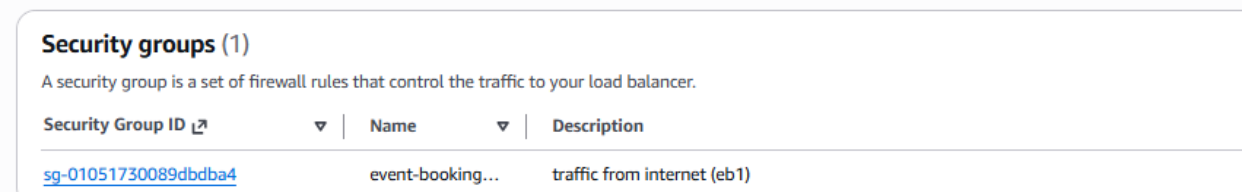
Key pair name
-

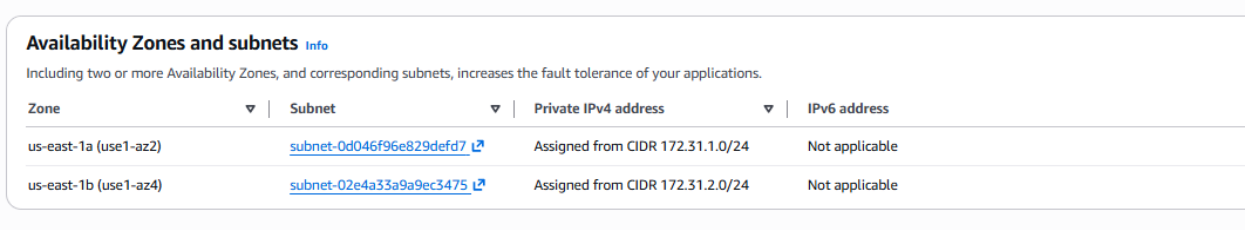
Request Spot Instances
No

View details in the launch template console

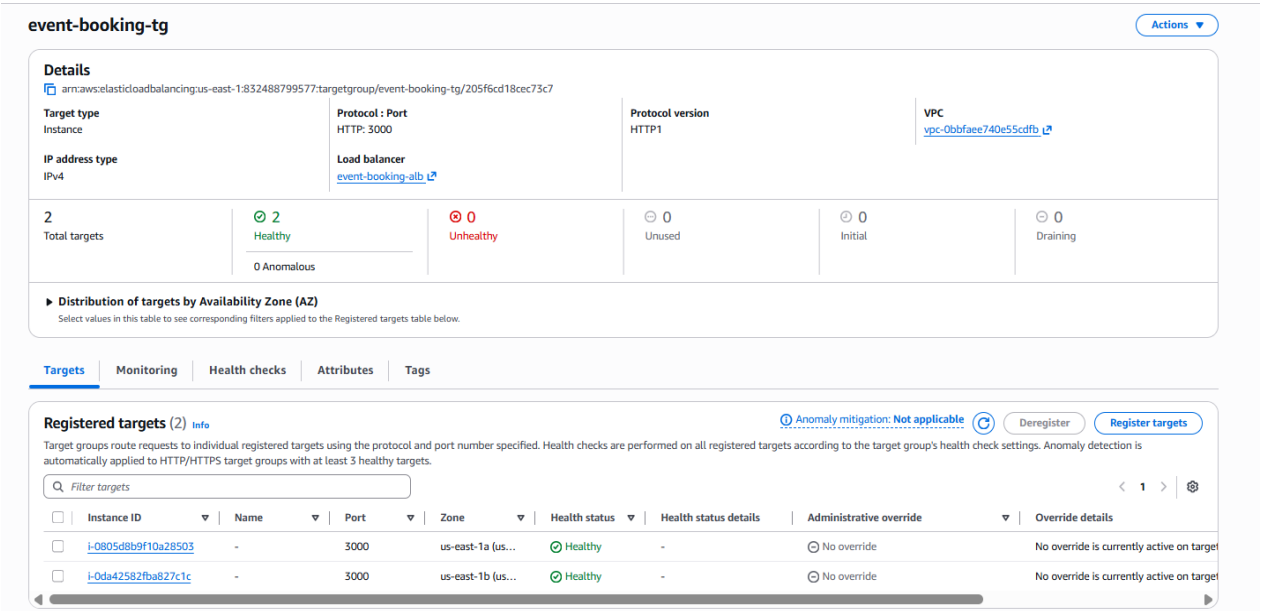
- **ALB**

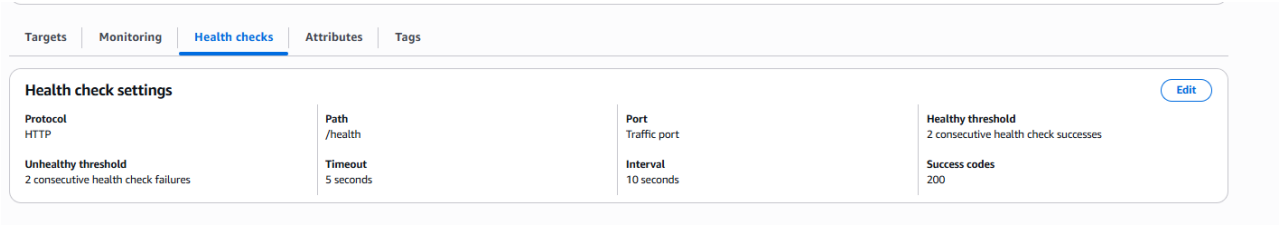




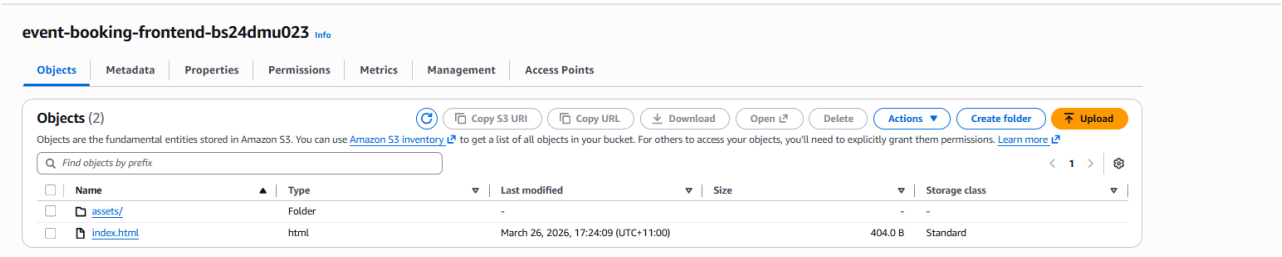


- **Target Group Healthy Instances**




Targets
Monitoring
Health checks
Attributes
Tags
Health check settings
Edit
Protocol
HTTP
Path
/health
Port
Traffic port
Healthy threshold
2 consecutive health check successes
Unhealthy threshold
2 consecutive health check failures
Timeout
5 seconds
Interval
10 seconds
Success codes
200

- **S3 frontend**


event-booking-frontend-bs24dmu023 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
Access Points
Objects (2)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
assets/
Folder
-
-
-
index.html
html
March 26, 2026, 17:24:09 (UTC+11:00)
404.0 B
Standard

I. Application Testing Proof

Website: http://event-booking-frontend-noe-2026.s3-website-us-east-1.amazonaws.com/

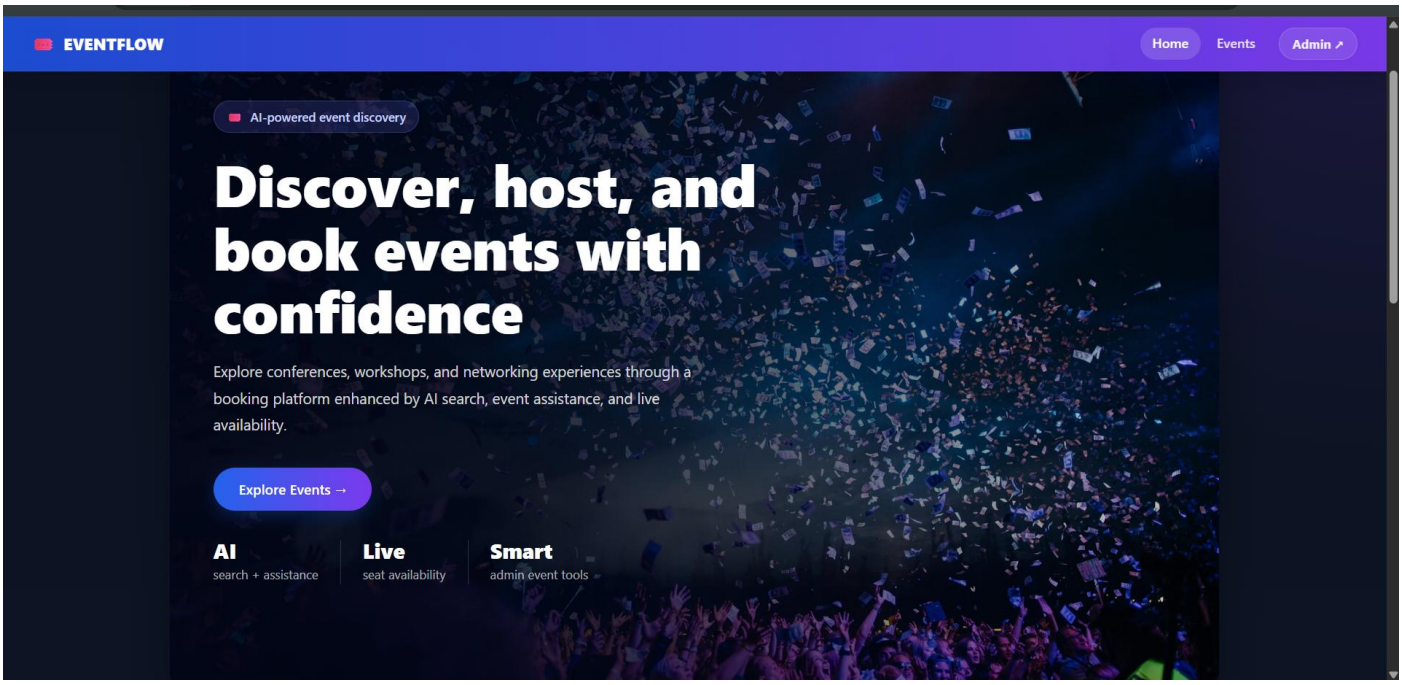
Video Overview:
The video demonstrates end-to-end application functionality, including:

- **User flow**: browsing events and making bookings
- **Booking constraints**: capacity enforcement
- **Admin flow:** authentication, event and booking management
- **AI features**: optional enhancements for search and content improvement

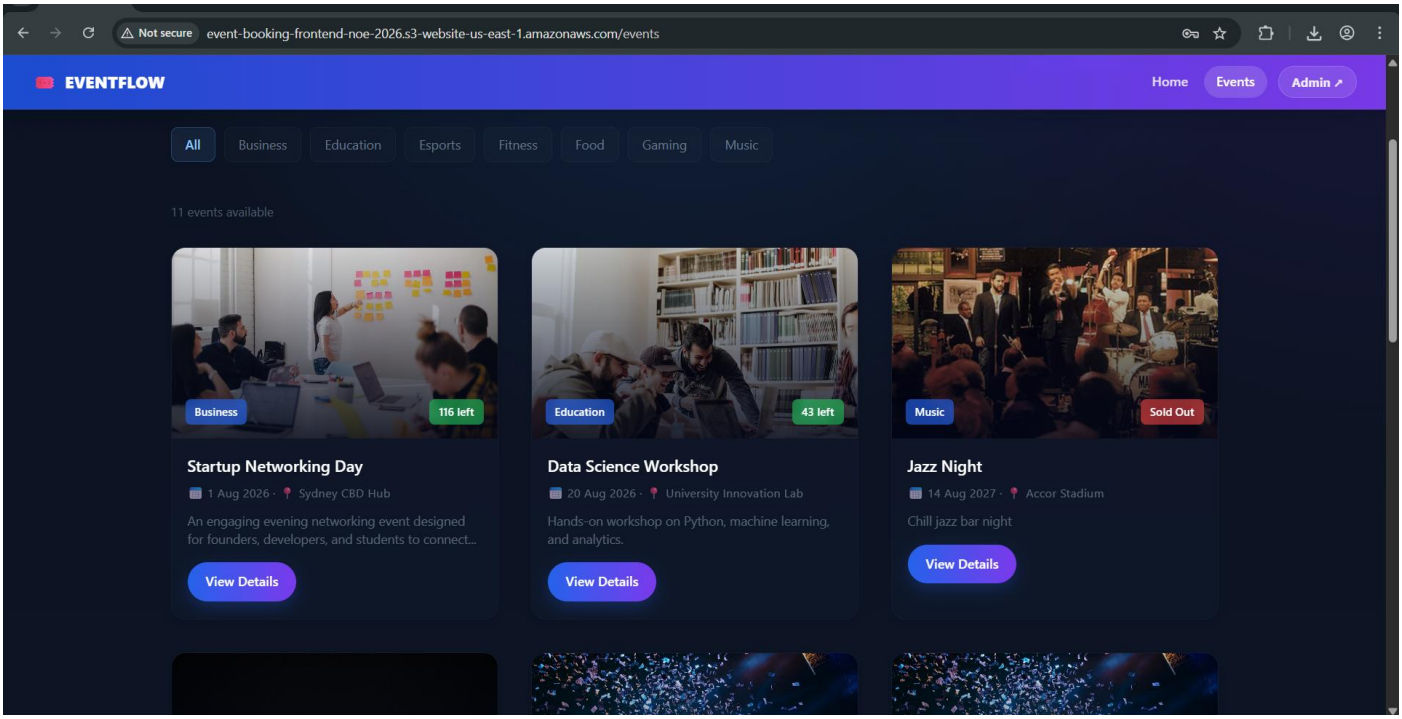
Website Video Proof: https://www.loom.com/share/1758bd63182b4588806a31ad396faddb

Note:
- Booking **works independently of AI**. AI is an **optional enhancement** for content and search.
- Aishayah performed QA testing and provided application screenshots.

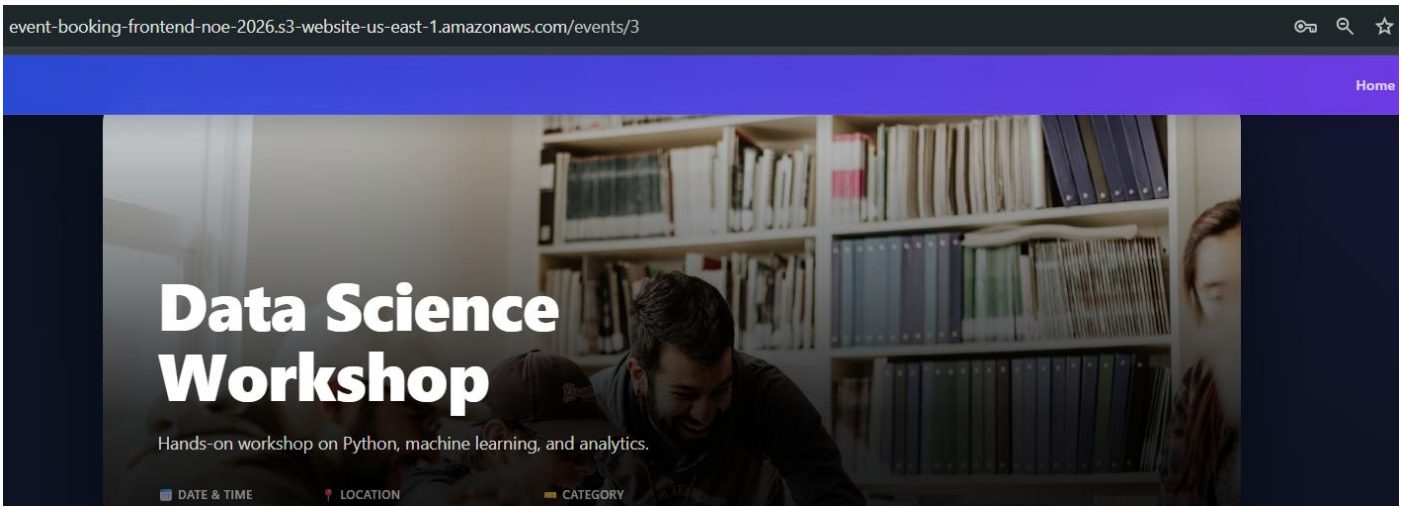
1. Homepage

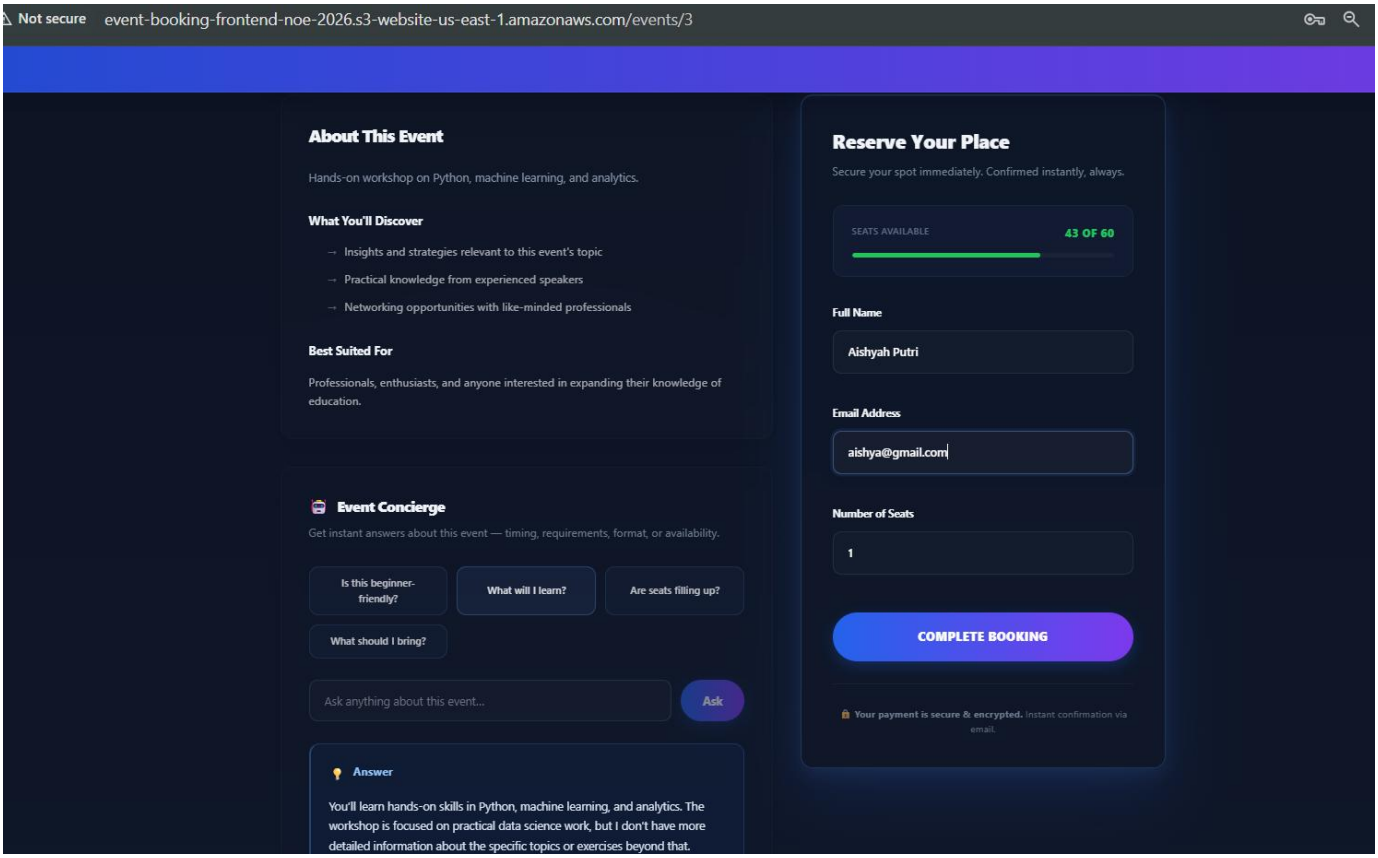


2. Event List Page

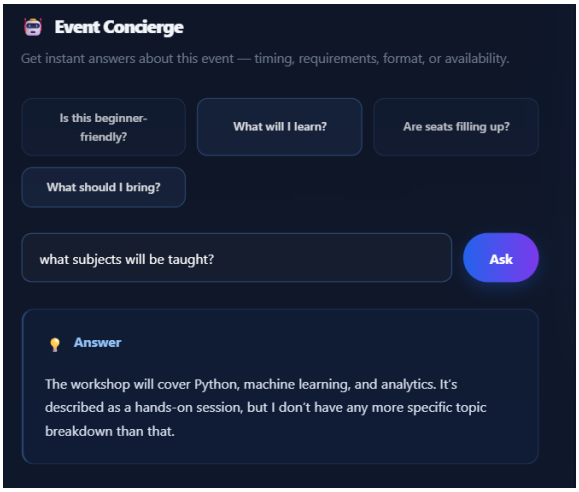


3. Event Detail Page (id = 3)

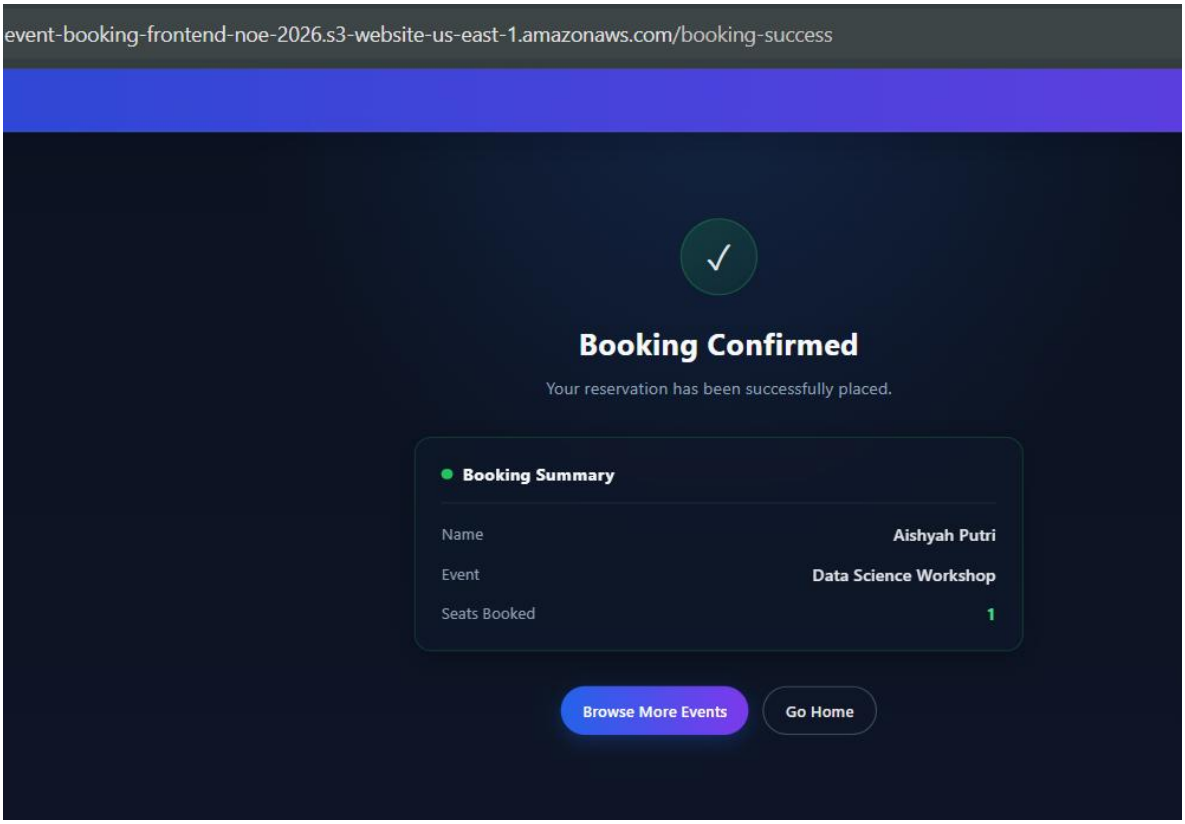




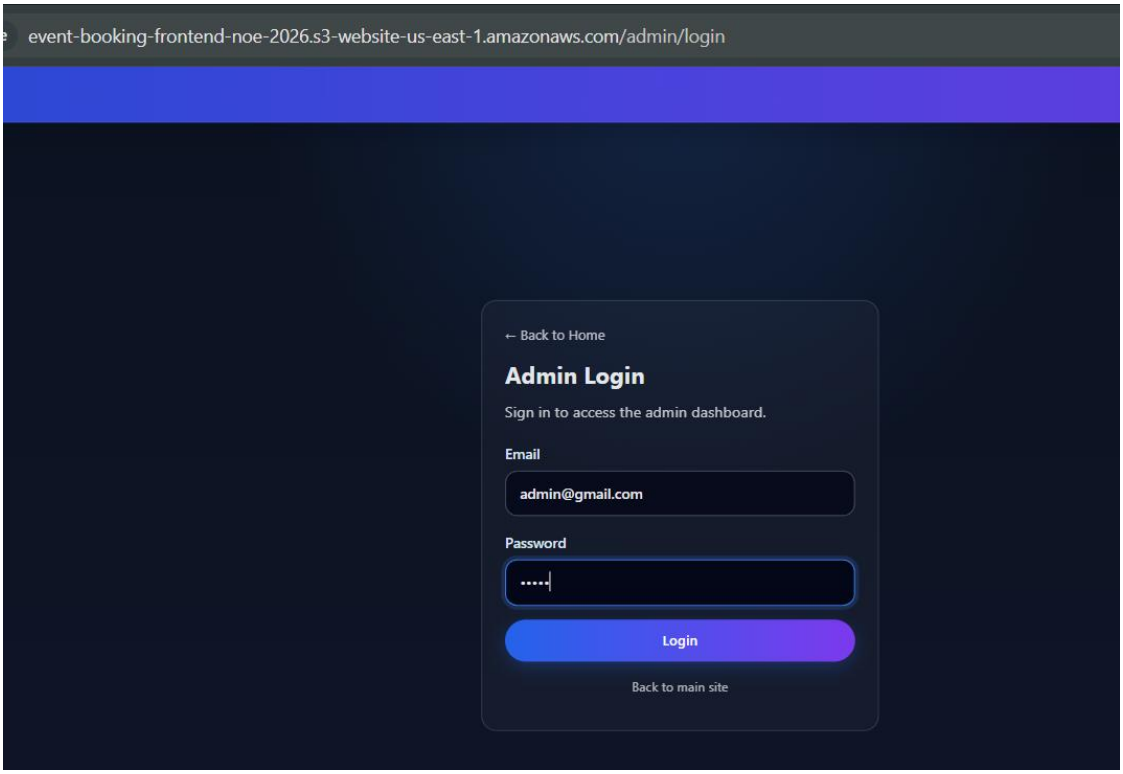
3.5. AI chatbot feature



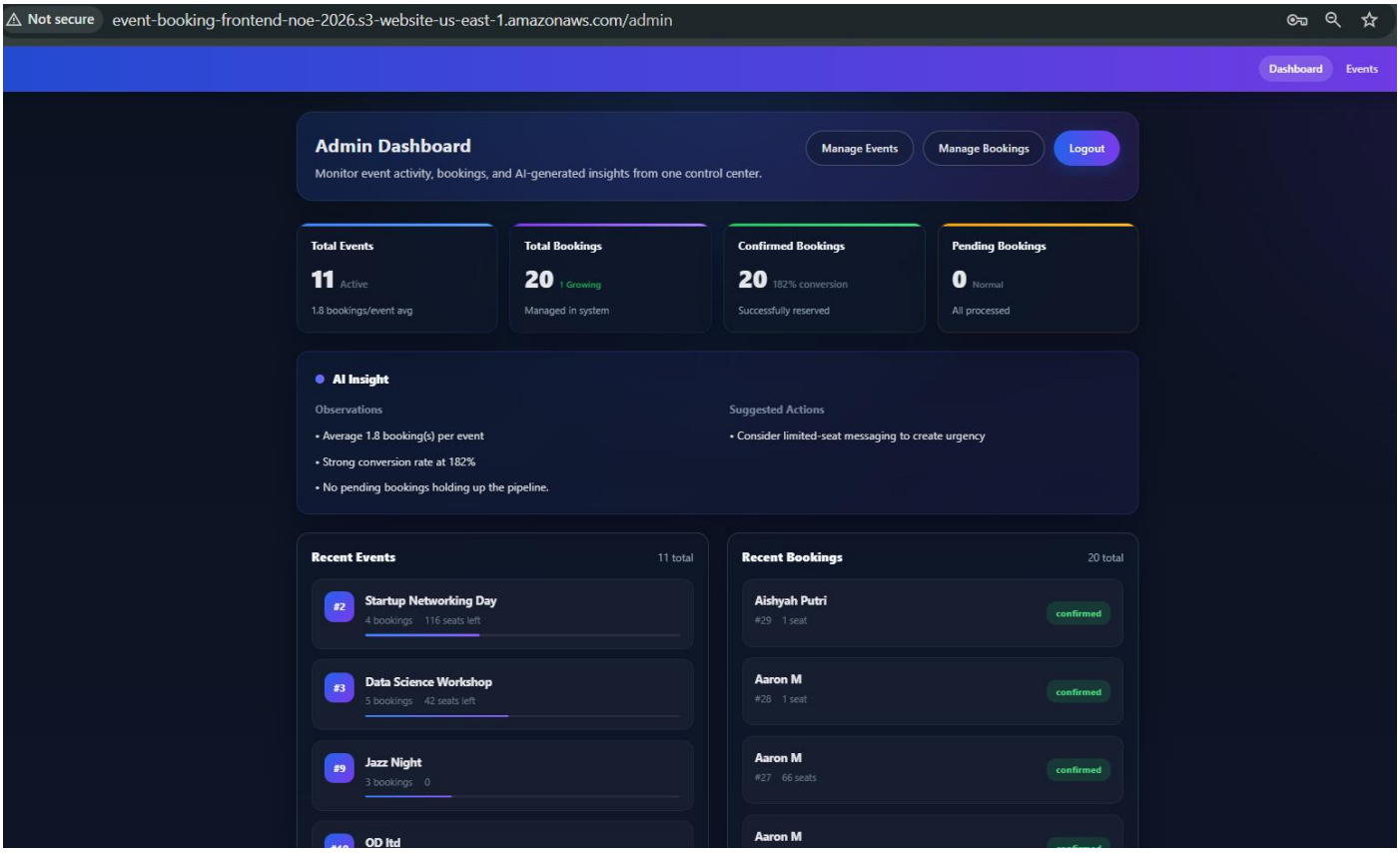
4. Booking Success Page



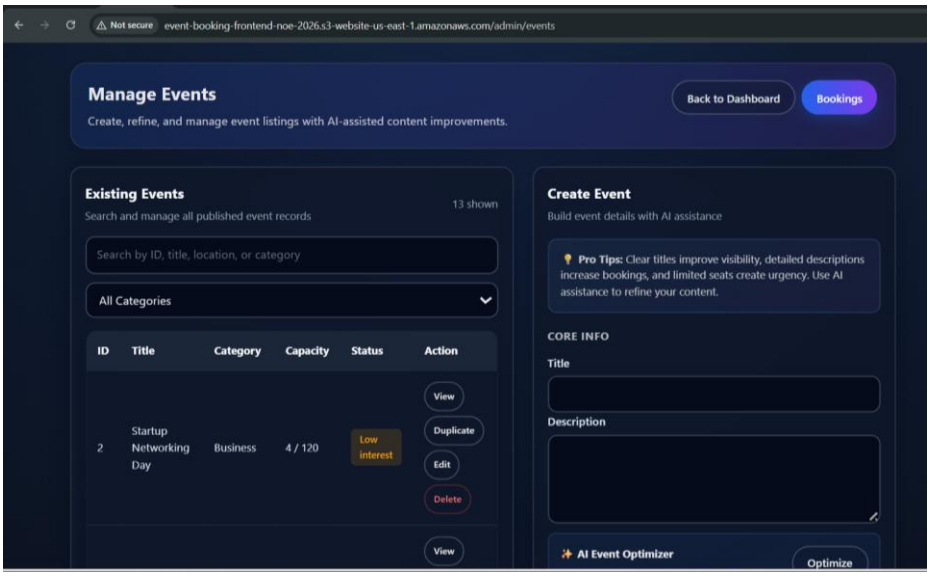
5. Admin Login Page [password: 12345; (encrypted in DB)]



6. Admin Dashboard Page



7. Admin Events Page



8. Admin Events Page (new event created)

AI suggestions applied successfully!

Existing Events

Search and manage all published event records

12 shown

Search by ID, title, location, or category

All Categories

ID	Title	Category	Capacity	Status	Action
2	Startup Networking Day	Business	4 / 120	Low interest	View, Duplicate, Edit, Delete
3	Data Science Workshop	Education	18 / 60	Low interest	View, Duplicate, Edit, Delete

Create Event

Build event details with AI assistance

Pro Tips: Clear titles improve visibility, detailed descriptions increase bookings, and limited seats create urgency. Use AI assistance to refine your content.

Event Quality Score
95/100

CORE INFO

Title

Wedding Events

Description

Celebrate your special day with beautifully planned wedding events designed to create unforgettable memories for you and your guests.

AI Event Optimizer
Improve title, description & category

Optimize

Suggested Tags

wedding celebration event ceremony

LOGISTICS

Date and Time

31-03-2027 00:00

Location

Olympic Park

CAPACITY

Category

Wedding

Total Capacity

1

Available Capacity

1

MEDIA

Image URL

https://event-booking-uploads-noe-2026.s3.us-east-1.amazo

Or upload image

Choose file wedding-cloud.jpeg

PDF Attachment URL

https://example.com/document.pdf

Or upload PDF

Choose file No file chosen

Create Event

Not secure event-booking-frontend-noe-2026.s3-website-us-east-1.amazonaws.com/events

Rap Song Travis

1 Apr 2026 · Bosnia

A focused event centered on rap music inspired by Travis, featuring energetic lyrics, modern be...

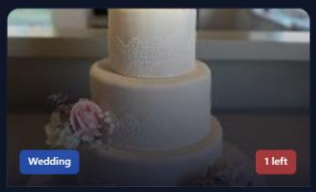
View Details

italian food

23 Apr 2027 · Sydney

An event celebrating the rich flavors and of Italian cuisine, featuring classic dish...

View Details



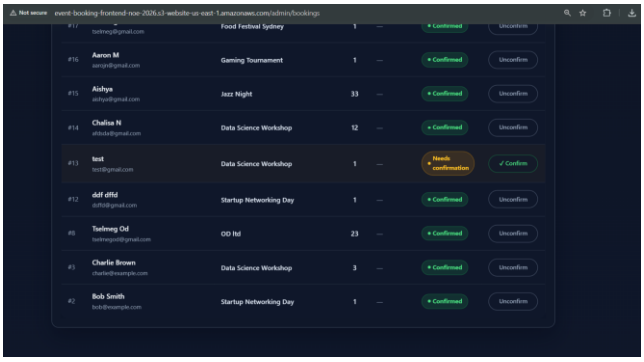
Wedding Events

31 Mar 2027 · Olympic Park

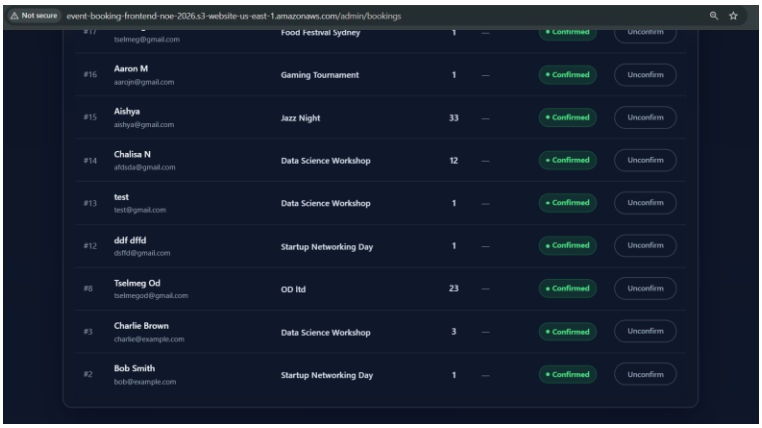
Celebrate your special day with beautifully planned wedding events designed to create unforgett...

View Details

9. Admin Bookings Page



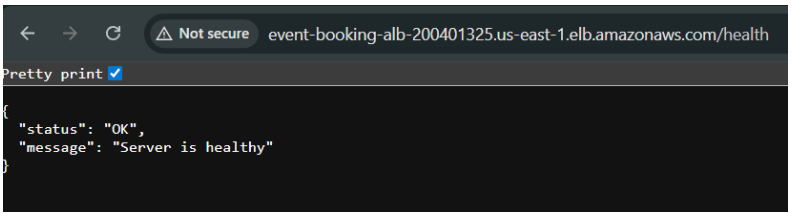
10. Admin Bookings Page (status=confirmed for id =13)



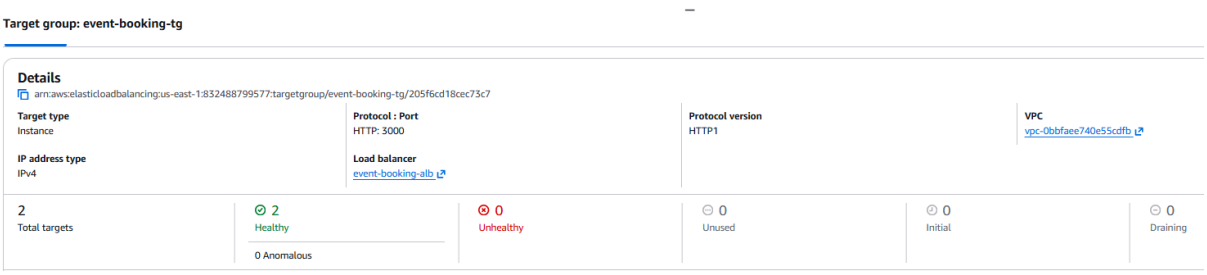
J. Mandatory Testing

1. High Availability Test

Step 1 — Baseline Verification

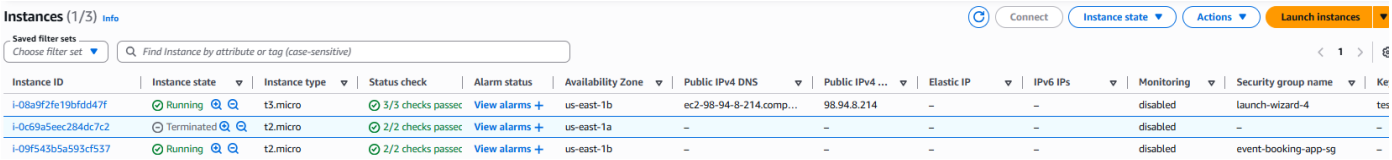


The system was operational prior to testing, with the application accessible via the ALB endpoint.



The target group showed two EC2 instances in a healthy state, confirming that traffic was being correctly distributed across multiple backend instances.

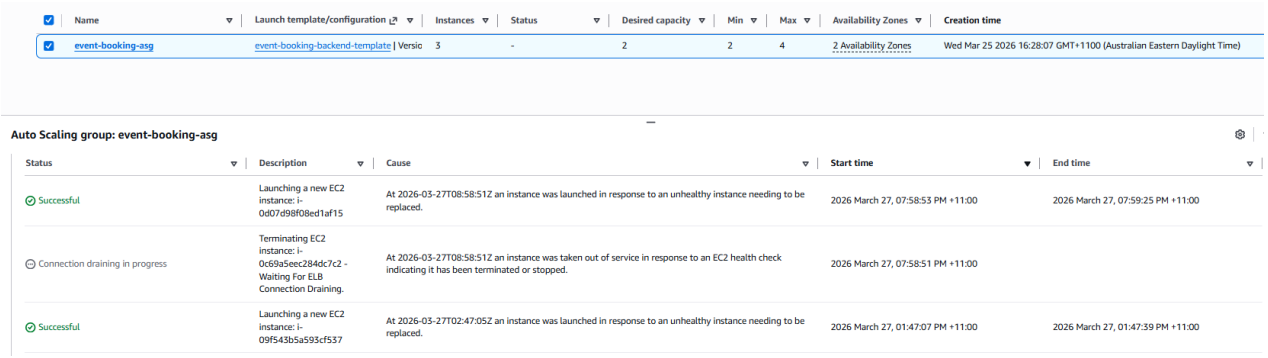
Step 2 — Simulating Instance Failure



One EC2 instance was intentionally terminated from the Auto Scaling Group to simulate a failure scenario.

This allowed observation of how the system responds to instance loss under load-balanced conditions.

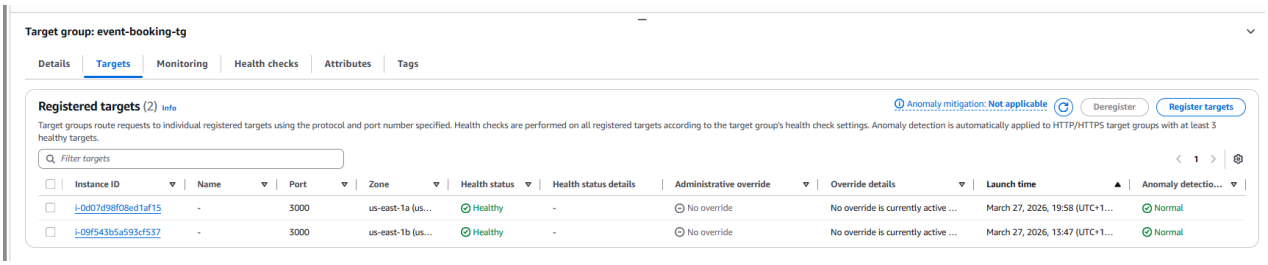
Step 3 — Auto Scaling Replacement



After the instance was terminated, the Auto Scaling Group automatically launched a new EC2 instance to maintain the desired capacity.

This demonstrates the system's ability to recover from instance failure without manual intervention.

Step 4 — Instance Becomes Healthy



The newly launched instance successfully passed health checks and was registered as healthy in the target group.

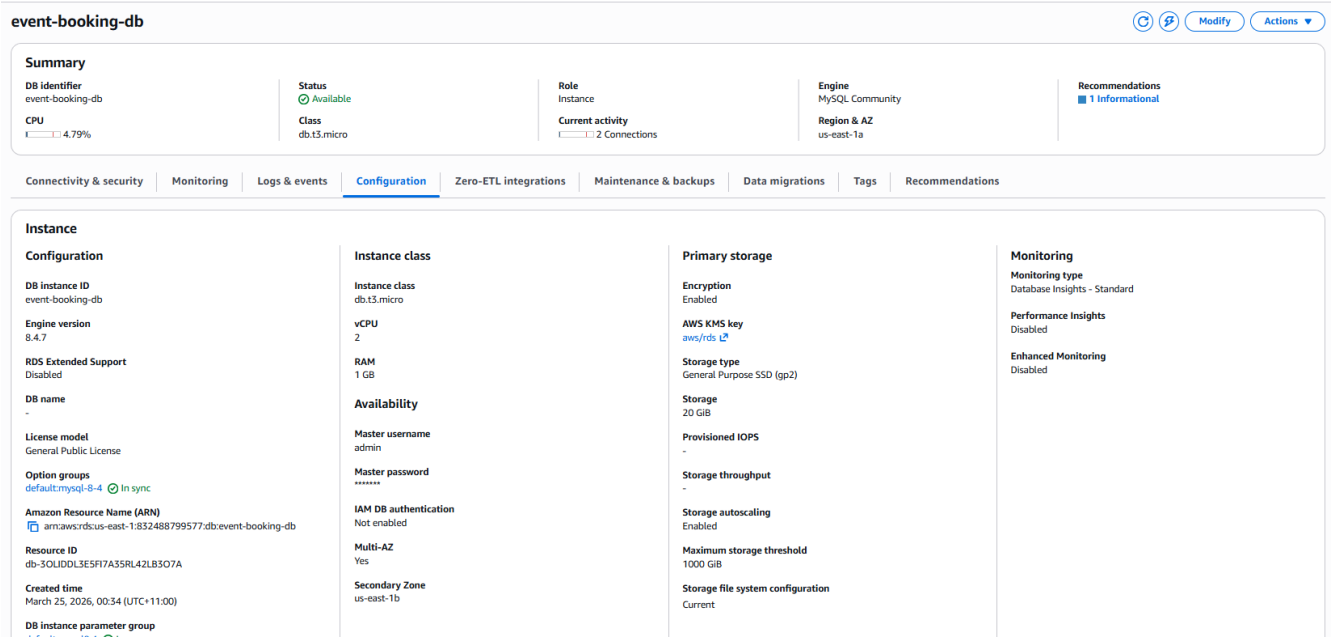
Traffic was automatically distributed across available instances, ensuring uninterrupted application availability.

Step 5 — Application Availability Verification

The application remained fully functional during the failure, as booking operations continued to succeed without disruption.

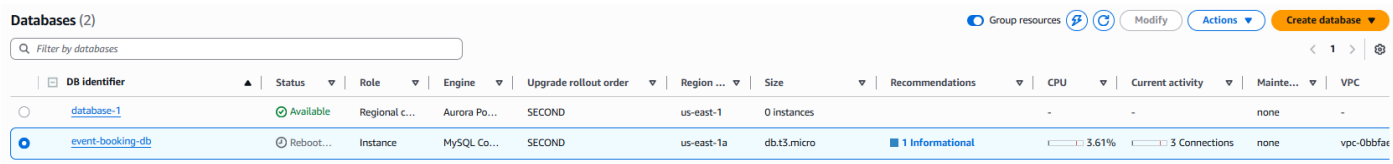
2. Database Failover Test

Step 1 — Baseline Verification



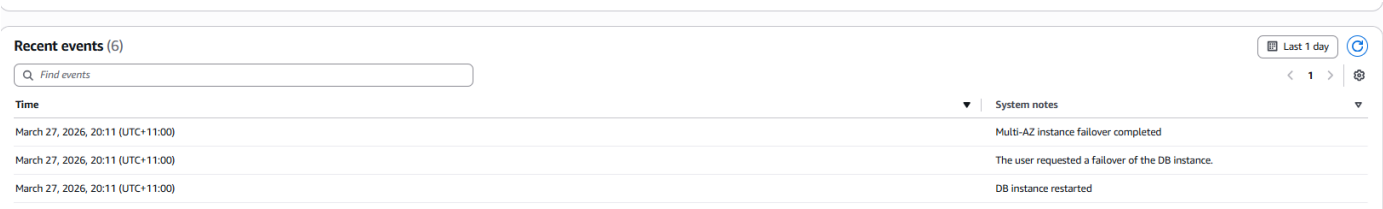
The application was verified to be operational prior to database failover. The RDS instance was in an available state with Multi-AZ enabled, ensuring redundancy across Availability Zones.

Step 2 — Trigger Database Failover



A manual failover was triggered by rebooting the RDS instance with the failover option enabled.

Step 3 — Failover Verification



The RDS event logs confirm that a failover was successfully triggered, where the standby instance was promoted to primary.

The database instance was restarted and failover was completed without manual intervention, demonstrating high availability of the database layer.

The application remained accessible during and after the failover.

Step 4 — Application Availability Verification

The application remained fully functional during the failure, as booking operations continued to succeed without disruption.

K. Step by Step Proof of Cloud Deployment

Step 1 — Create IAM Role for EC2 (Session Manager, no SSH)

---

1. Go to IAM

- Open AWS Console
- Search **IAM**
- Click **Roles** (left sidebar)

---

2. Create Role

- Click **Create role**

Select:

- **Trusted entity type** → AWS service
- **Use case** → EC2

Click **Next**

---

3. Attach permissions

Search and add:

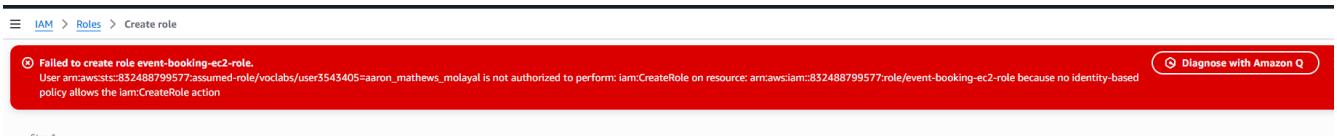
AmazonSSMManagedInstanceCore

Click **Next**

---

4. Name the role: event-booking-ec2-role

Click **Create role**



Note: The AWS lab account restricts IAM role creation (iam:CreateRole permission not granted). Therefore, LabRole or LabInstanceRole is used for EC2 instances in accordance with the environment constraints.

(Step 1 has been incorporated to ensure alignment with the assessment rubric)

Step 2 — Create a VPC

1. Go to VPC

- Search **VPC**
- Click **Your VPCs**
- Click **Create VPC**

---

2. Choose settings

Select: VPC only

---

3. Configure VPC

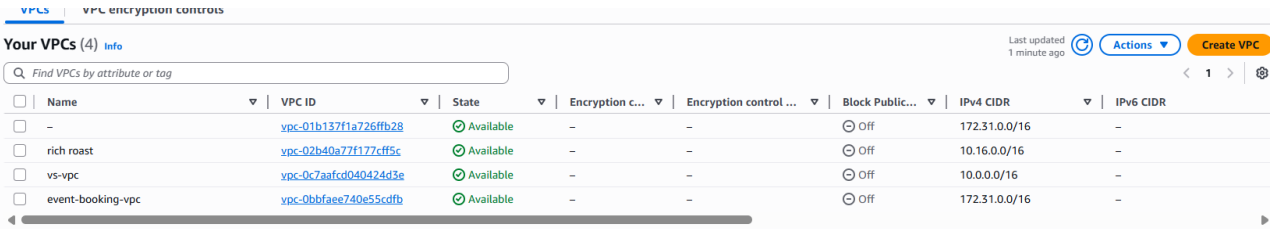
Use fixed values (do not change later):

Name: event-booking-vpc

IPv4 CIDR: 172.31.0.0/16

Tenancy: Default

Click **Create VPC**



Step 3 — Create Subnets across 2 Availability Zones

1. Go to Subnets

- Open **VPC Dashboard**
- Click **Subnets**
- Click **Create subnet**

---

2. Select VPC

Choose:

event-booking-vpc

---

3. Create 6 subnets

Use consistent structure (do not improvise names or CIDRs):

Public Subnets (for ALB)

Name: public-subnet-1a

AZ: us-east-1a

CIDR: 172.31.1.0/24

Name: public-subnet-1b

AZ: us-east-1b

CIDR: 172.31.2.0/24

Private App Subnets (for EC2)

Name: private-app-subnet-1a

AZ: us-east-1a

CIDR: 172.31.3.0/24

Name: private-app-subnet-1b

AZ: us-east-1b

CIDR: 172.31.4.0/24

Private DB Subnets (for RDS)

Name: private-db-subnet-1a

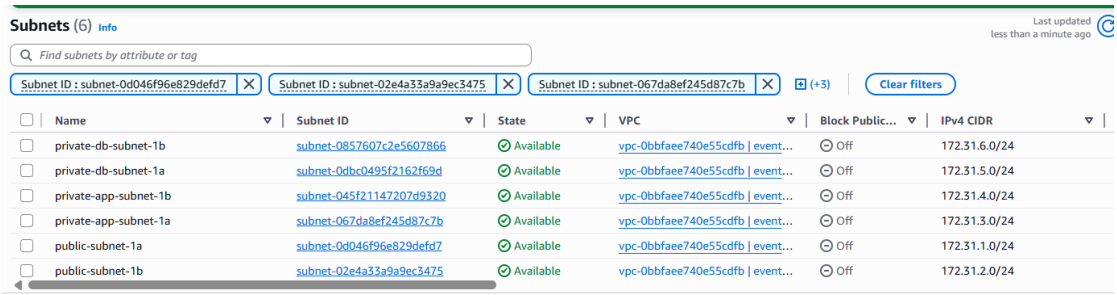
AZ: ap-southeast-2a

CIDR: 172.31.5.0/24

Name: private-db-subnet-1b

AZ: ap-southeast-2b

CIDR: 172.31.6.0/24



Step 4 — Attach Internet Gateway (IGW)

1. Create Internet Gateway

- Go to **VPC Dashboard**
- Click **Internet Gateways**
- Click **Create internet gateway**

Enter:

Name: event-booking-igw

Click **Create**

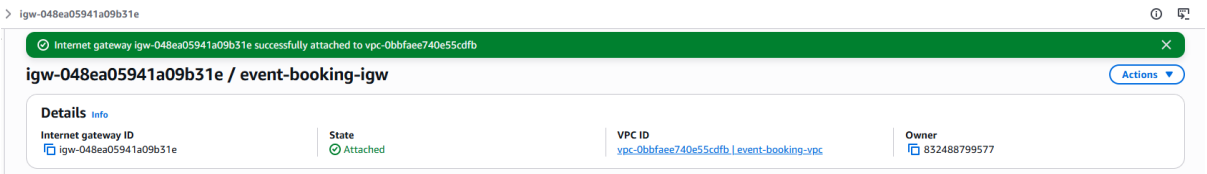
---

2. Attach IGW to VPC

- Select the created IGW
- Click **Actions → Attach to VPC**
- Choose:

event-booking-vpc

- Click **Attach**



Step 5 — Create NAT Gateway

1. Go to NAT Gateway

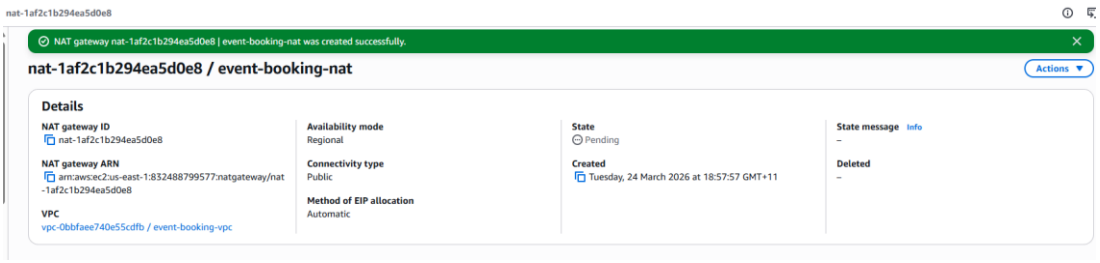
- Open **VPC Dashboard**
- Click **NAT Gateways**
- Click **Create NAT Gateway**

---

2. Configure NAT Gateway
Select:
Name: event-booking-nat
Vpc: event-booking-vpc
Connectivity type: Public

---

3. Create
Click **Create NAT Gateway**
Wait until status becomes:
Available



Step 6 — Create Route Tables

1. Go to Route Tables

- Open VPC Dashboard
- Click Route Tables
- Click Create route table

---

2. Create Public Route Table
Name: event-booking-public-rt
VPC: event-booking-vpc
Add route:
0.0.0.0/0 → Internet Gateway (event-booking-igw)
Associate with:

- public-subnet-a
- public-subnet-b

---

3. Create Private App Route Table
Name: private-app-rt
VPC: event-booking-vpc
Add route:
0.0.0.0/0 → NAT Gateway (event-booking-nat)
Associate with:

- private-app-subnet-a
- private-app-subnet-b

---

4. Create Private DB Route Table

Name: private-db-rt

VPC: event-booking-vpc

Routes:

local only (default)

Associate with:

- private-db-subnet-a
- private-db-subnet-b

☐	event-booking-public-rt	rtb-0305ee737cc078672	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577
☐	event-booking-private-app-rt	rtb-060c4af18e828a922	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577
☐	event-booking-private-db-rt	rtb-0292353ed58564e37	2 subnets	–	No	vpc-0bbfaee740e55cdfb | event...	832488799577

Step 7 — Create Security Groups

1. Go to Security Groups

- Open **VPC Dashboard** or **EC2 Dashboard**
- Click **Security Groups**
- Click **Create security group**

Use the same VPC:

event-booking-vpc

7.1 Create ALB Security Group

Name

event-booking-alb-sg

VPC

event-booking-vpc

Inbound rules

HTTP 80 Source: 0.0.0.0/0

HTTPS 443 Source: 0.0.0.0/0 (only if using HTTPS)

Outbound rules

Leave as default

ALB forwards traffic to the App layer.

7.2 Create App Security Group

Name

event-booking-app-sg

VPC

event-booking-vpc

Inbound rules

Custom TCP 3000 Source: event-booking-alb-sg

Note: Use your backend app port only from the ALB security group. (My port is 3000)

Do not allow: 0.0.0.0/0

Outbound rules

Leave as default. App SG forwards traffic to database

7.3 Create DB Security Group

Name
event-booking-db-sg
VPC
event-booking-vpc
Inbound rules
MySQL/Aurora 3306 Source: event-booking-app-sg
Only the App layer should reach the DB.
Outbound rules
Leave as default.

☐	–	sg-01051730089dbdba4	event-booking-alb-sg	vpc-0bbfaee740e55cdfb	traffic from internet (eb1)	832488799577
☐	–	sg-007cb4e791950ef9a	event-booking-db-sg	vpc-0bbfaee740e55cdfb	traffic from application	832488799577
☐	–	sg-060f57539b77522fd	event-booking-app-sg	vpc-0bbfaee740e55cdfb	traffic from alb	832488799577

Step 8 — Create DB Subnet Group

1. Go to RDS

- Search **RDS**
- Click **Subnet groups**
- Click **Create DB subnet group**

2. Configure DB Subnet Group
Name: event-booking-db-subnet-group
Description: DB subnets for private RDS
VPC: event-booking-vpc

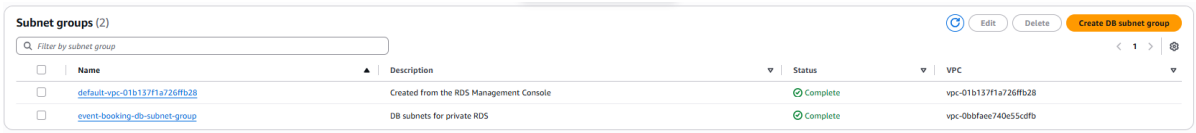
---

3. Add Subnets
Availability zones:
us-east-1a, us-east-1b

Select subnets:
private-db-subnet-1a
private-db-subnet-1b

---

4. Create
Click **Create**



Step 9 — Create RDS MySQL (Private, Multi-AZ)

1. Go to RDS

- Open **RDS**
- Click **Databases**

- Click **Create database**

---

2. Choose engine
Engine: MySQL
Creation method: Full configuration
Template: Dev/Test

---

3. Availability (FIXED)
Deployment option: Multi-AZ DB instance (2 instances)

---

4. Configure settings
DB instance identifier: event-booking-db
Master username: admin
Credential management: Self managed
Master password: password

---

5. Instance configuration (OPTIMIZED)
Burstable classes (includes t classes)
Instance class: db.t3.micro

---

6. Storage
Storage type: General Purpose SSD (gp3)
Allocated storage: 20 GB

---

7. Connectivity (CRITICAL)
Compute resource: Don't connect to EC2
VPC: event-booking-vpc
DB subnet group: event-booking-db-subnet-group
Public access: NO

---

8. Security
VPC security group: event-booking-db-sg
Required rule inside SG:
Allow MySQL (3306) from app-sg ONLY

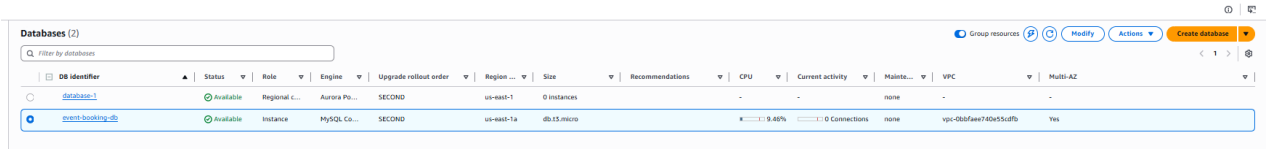
---

9. Monitoring
Database Insights: Standard
Performance Insights: Enabled

---

10. Create

- Click **Create database**
- Wait until status: Available



Note (Design decision):
RDS is created first so the application can be configured with the final database endpoint from the start, avoiding reconfiguration later. A temporary EC2 instance is used only to securely initialize the private database before deploying the actual backend instances.

Step 10 — Initialize Database (RDS)

1. Launch Temporary Access Instance

- Go to EC2
- Click **Launch instance**

Configure:

- Name: temp-db-access-instance
- AMI: Ubuntu
- Instance type: t3.micro
- Key pair: Proceed without key pair
- Network:
  - VPC: event-booking-vpc
  - Subnet: private-app-subnet-1a
  - Auto-assign Public IP: Disabled
- Security Group:
  - Use **event-booking-app-sg**
- IAM Role: LabInstanceProfile [Advance details]

Launch instance

---

2. Connect Using Session Manager

- Go to EC2 → Instances
- Select the instance
- Click **Connect**
- Choose **Session Manager**
- Click **Connect**

---

3. Install MySQL Client

Run:

sudo apt update

sudo apt install mysql-client -y

---

4. Upload .sql File to EC2 (use exact URL)

cd ~

wget https://raw.githubusercontent.com/netnetnet023/database-not-a-project/main/eb_database.sql

Note: cd ~ only if you get write permission denied error

---

5. Import .sql File into RDS

- Go to RDS → Databases →event-booking-db
- Connectivity and Security →Copy the **Endpoint**

Run(Session Manager):

mysql -h <RDS_ENDPOINT> -u admin -p

CREATE DATABASE event_booking;

EXIT;

mysql -h <RDS_ENDPOINT> -u admin -p event_booking < eb_database.sql

password: *password*

This will:

- create tables
- insert data
- apply full schema from file

---

6. Verify Database Setup

mysql -h <RDS_ENDPOINT> -u admin -p

Then:

USE event_booking;

SHOW TABLES;

SELECT * FROM events;

Mm

```
Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE event_booking;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-------------------------+
| Tables_in_event_booking |
+-------------------------+
| admins                  |
| bookings                |
| events                  |
+-------------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM admins;
+----+-------+----------------+--------------------------------------------------------------+
| id | name  | email          | password_hash                                                |
+----+-------+----------------+--------------------------------------------------------------+
|  1 | admin | admin@gmail.com | $2b$10$snvyMOI4BSzGOGPOm1xvX.LguC5axHcOgS81cONHIyohCvgmNfmly |
+----+-------+----------------+--------------------------------------------------------------+
1 row in set (0.00 sec)

mysql> exit;
```

---

7. Security Group Check(optional)

Ensure RDS security group allows:

RDS → Databases

- Type: MySQL (3306)
- Source: **event-booking-app-sg**

Ec2→instances→**temp-db-access-instance**→Security→inbound rules→event-booking-app-sg

▼ Inbound rules

Filter rules

urity group rule ID	Port range	Protocol	Source	Security groups	Description
-08d35ea a7b91a07b	3000	TCP	sg-01051730089dbdba4	event-booking-app-sg	-

---

Step 11 — Create EC2 Launch Template

---

1. Go to EC2

- Search **EC2**
- Click **Launch Templates**
- Click **Create launch template**

---

2. Configure Launch Template

Set:

- **Launch template name:** event-booking-backend-template
- **Template version description:** Backend template for Auto Scaling Group instances

---

3. Network

Subnet: Private-app-subnet-1a

Availability Zone: us-east-1a

Security group: event-booking-app-sg

3. Select AMI and Instance Type

Choose:

- **AMI:** Ubuntu
- **Instance type:** t2.micro

---

4. Configure IAM Role

Under **Advanced details**:

IAM instance profile: LabInstanceProfile

---

5. Key pair: Proceed without key pair

6. Add User Data

In **Advanced details → User data**, type:

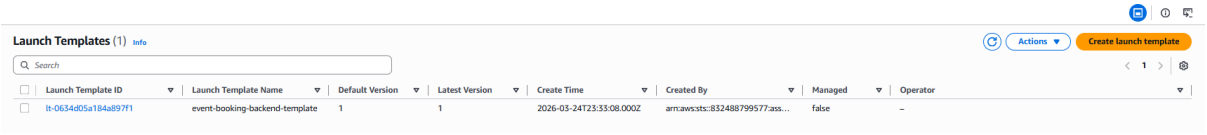
#!/bin/bash

```
set -euxo pipefail

exec > >(tee /var/log/user-data.log | logger -t user-data) 2>&1

apt-get update -y
apt-get install -y curl git unzip

curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

npm install -g pm2

mkdir -p /home/ubuntu/app
cd /home/ubuntu/app

git clone -b main https://github.com/netnetnet023/backend-for-deployment-not-a-project.git
cd "/home/ubuntu/app/backend-for-deployment-not-a-project/updated_backend – Copy"

cat > .env <<EOF
PORT=3000
DB_HOST=event-booking-db.xxxxxxxxxxxx.us-east-1.rds.amazonaws.com
DB_PORT=3306
DB_USER=admin
DB_PASSWORD=password
DB_NAME=event_booking
JWT_SECRET=supersecretkey123
EOF

npm install

chown -R ubuntu:ubuntu /home/ubuntu/app

sudo -u ubuntu pm2 start src/server.js --name event-booking-backend
sudo -u ubuntu pm2 save

env PATH=$PATH:/usr/bin pm2 startup systemd -u ubuntu --hp /home/ubuntu
systemctl enable pm2-ubuntu
systemctl start pm2-ubuntu
```

---

7. Create Launch Template

- Review settings
- Click **Create launch template**



Step 12 — Verify Session Manager Readiness

1. Launch Test Instance

- Go to **EC2 → Launch Templates**
- Select **event-booking-backend-template**
- Click **Actions → Launch instance from template**
- Launch **1 temporary instance**

2. Verify Session Manager Access

- Go to **EC2 → Instances**
- Select the instance (not the temporary instance)
- Click **Connect → Session Manager**

Confirm:

- Session Manager works
- No key pair is required

3. Verify Backend Startup

Inside the instance, run:

sudo -i -u ubuntu

pm2 list

Confirm the backend process is running.

4. Verify Health Endpoint

Run:

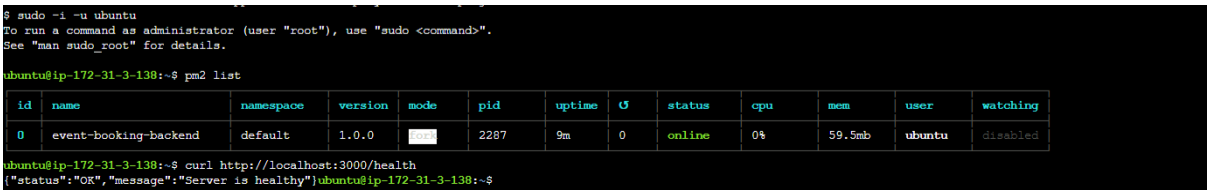
curl http://localhost:3000/health

Confirm the application responds successfully.

5. Confirm Security

- SSH port **22** must **not** be open
- access is only through **Session Manager**

Note: now you can stop temp-db-access-instance



Step 13 — Create Target Group

1. Go to Target Groups

- Go to **EC2 → Target Groups**
- Click **Create target group**

---

2. Choose Target Type

Set:

- **Target type**: Instances
- **Target group name**: event-booking-tg
- **Protocol**: HTTP
- **Port**: 3000

---

3. Select VPC

Set:

- **VPC**: event-booking-vpc

---

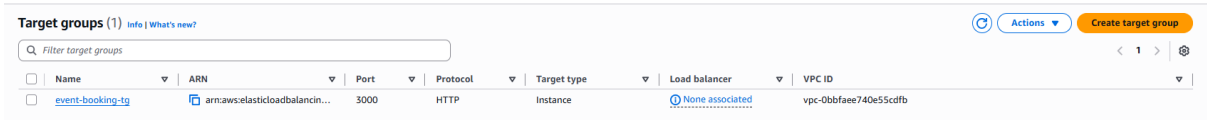
4. Configure Health Check

Set:

- **Protocol**: HTTP
- **Path**: /health

Advance health check settings:

- **Port**: traffic port
- Healthy threshold: 2
- Unhealthy threshold: 2
- Timeout: 5s
- Interval: 10s



Step 14 — Create Application Load Balancer (ALB)

---

1. Go to Load Balancers

- Go to **EC2 → Load Balancers**
- Click **Create Load Balancer**
- Choose **Application Load Balancer**

---

2. Basic Configuration

Set:

- **Name**: event-booking-alb
- **Scheme**: Internet-facing
- **IP type**: IPv4

---

3. Network Mapping

Set:

- **VPC**: event-booking-vpc
- **Availability Zones**:
  - public-subnet-1a
  - public-subnet-1b

---

4. Security Group

Attach:

- **event-booking-alb-sg**

---

5. Listener Configuration

Set:

- **Protocol**: HTTP
- **Port**: 80

Default action:

- **Forward to** → event-booking-tg

---

6. Create Load Balancer

- Click **Create Load Balancer**

---

7. Go to event-booking-tg ->Register Targets

- Select backend instance
- Port: 3000
- Click Include as pending below
- Click Register pending targets

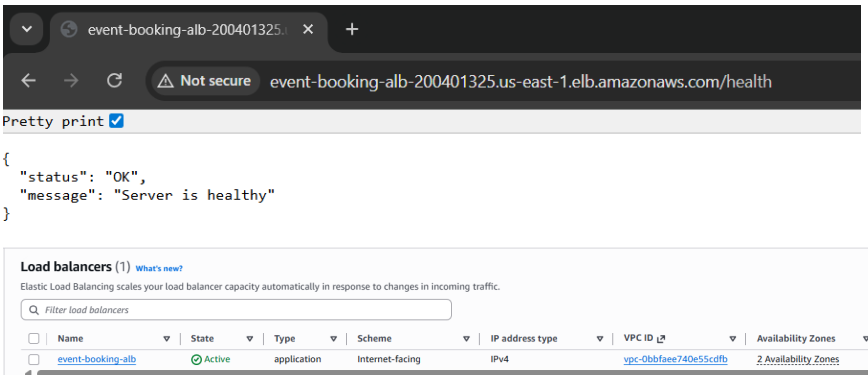
---

8. After creation

Wait 1–2 minutes

Then:

- Copy **DNS name** of ALB
- Open in browser: http://<ALB-DNS>/health



Step 15 — Create Auto Scaling Group (ASG)

1. Go to Auto Scaling Groups

- Go to **EC2 → Auto Scaling Groups**
- Click **Create Auto Scaling Group**

---

2. Basic Configuration

Set:

- **Name**: event-booking-asg
- **Launch template**: event-booking-backend template

---

3. Network Mapping

Set:

- **VPC**: event-booking-vpc
- **Availability Zones and subnets**:
  - private-app-subnet-1a
  - private-app-subnet-1b

---

4. Load Balancing

Set:

- Select **Attach to an existing load balancer**
- Choose **Application Load Balancer**
- Select event-booking-tg

---

5. Health Checks

Set:

- **Health check type**: EC2 + ELB
- **Health check grace period**: 60–120 seconds

---

6. Group Size and Scaling

Set:

- **Desired capacity**: 2
- **Minimum capacity**: 2
- **Maximum capacity**: 4

---

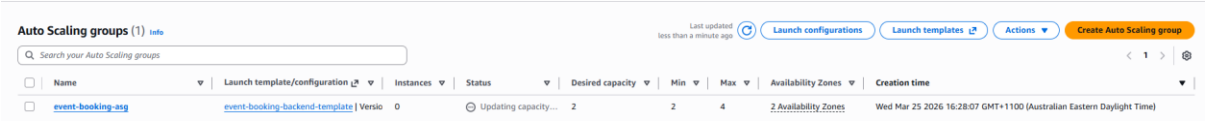
7. Additional Settings

- Leave defaults unless required
- Ensure **instance protection** is OFF

---

8. Review and Create

- Click **Create Auto Scaling Group**



9. Check EC2 instances

Go to **EC2 → Instances**

Confirm:

- **2 instances** exist
- both are in **running** state
- both were launched by the **Auto Scaling Group**

---

10. Check target group health

Go to **EC2 → Target Groups**

- select your target group
- open the **Targets** tab

Confirm:

- **2 targets** are registered
- both show **Healthy**

---

11. Check ALB DNS

Go to **EC2 → Load Balancers**

- select your ALB
- copy the **DNS name**

Open in browser:

- http://<ALB-DNS> /health

Confirm:

- backend response loads
- no timeout
- no 5xx error

Step 17 - Test backend via ALB DNS (With Postman)

1. Core endpoints:

GET http://<ALB-DNS>/health	"status": "OK", "message": "Server is healthy"	Verifies ALB
GET http://<ALB-DNS>/events	Json list of events	Verifies RDS

For POST method

POST http://<ALB-DNS>/bookings
 	
Content-Type: application/json
	{
	"event_id": 3,
	"customer_name": "Rin Doe",
	"customer_email": "Rin@gmail.com",
	"seats_booked": 6,
	"booking_status": "pending"
	}
		{
		"message": "Booking created successfully",
		"bookingId": 10
		}
		
		Verifies writes

Step 18 - Create S3 bucket for frontend

Note: frontend code was update to make it production ready. npm run build was used. Which created dist folder that will be uploaded to s3

1.Build frontend

npm run build

This creates:
dist/

---

2. Create S3 bucket
Go:

- AWS → S3 → Create bucket

Settings:

- Bucket name: event-booking-frontend-<studentid>

Uncheck: Block all public access
Create Bucket

3.Upload build files
Open bucket → Upload
Upload **contents of dist/ (contents not the folder)**

```
dist/
  index.html
  assets/
```

Upload

---

4. Enable static hosting
Go:

- Properties → Static website hosting → Enable

Set:
Index document: index.html
Error document: index.html ← IMPORTANT (React routing fix)

---

5. Add bucket policy (CRITICAL)
Go:

- Permissions → Bucket policy

Paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicRead",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

---

6. Open frontend
Go to:

S3 → your bucket→Properties → Static website hosting → Endpoint

E.g.
http://event-booking-frontend-bs24dmu023.s3-website-us-east-1.amazonaws.com/

---

7. Final test

From S3 URL:

- open homepage
- go to events
- open event detail
- book event
- Go back to homepage
- Log into admin page { email: admin@gmail.com , password: 12345}
- Test adminbookings, adminevents and logout

L. AI Integration and System Enhancements

Overview

The system was enhanced with additional intelligent and content management features to **improve administrator workflow** and **user experience**. These features are integrated through backend APIs and extend the existing event booking platform **without modifying the core booking flow.**

These enhancements are not core to the booking system. They **improve usability**, **reduce manual effort**, and enhance user interaction.

The enhancements include:

- AI-assisted event content improvement
- AI-based event search and interaction
- Dashboard summaries for administrators
- Image and attachment upload support

---

1. AI Features

AI-Assisted Event Improvement (Admin)

Allows administrators to enhance event descriptions using AI-based processing.
This helps generate clearer and more structured content while reducing manual effort.

AI-Based Search and Interaction (User)

Allows users to interact with the platform more flexibly.
Supports:

- Event search using natural language input
- AI-assisted interaction for a specific event

This improves event discoverability and user experience.

AI-Based Dashboard Summary (Admin)

Provides summarized insights for administrators to quickly understand platform activity and key metrics.

---

2. Content and File Management Enhancements

Image Upload

Allows administrators to upload images for events, improving visual presentation on the frontend.

Attachment Upload

Supports uploading documents (e.g., PDFs) for events, enabling additional information to be shared with users.

---

3. Enhancement Endpoints

Admin Endpoints

/ai/improve-event	Improve event content
/ai-summary	Generate dashboard summary
/upload-image	Upload event image
/upload-attachment	Upload supporting document

Event Endpoints

/ai-search	Search events using natural language input
/:id/ai-chat	AI-assisted interaction for a specific event

---

4. Backend Integration

These features are implemented through the backend API layer.

Flow:

- Request received from frontend or admin panel
- Routed through controllers and middleware
- External AI services are used where required
- File uploads are handled via dedicated middleware
- Processed response returned to the client

This ensures all enhancements follow the existing system architecture.

---

5. System Support Components

To support these features, the following components were added:

- AI-related routes under admin and event modules
- Controllers for handling AI processing and file uploads
- Middleware for image and document uploads
- Storage utilities for managing uploaded files

These additions extend the system while maintaining the existing backend structure.

---

6. Value Added to the System

These enhancements improve the platform by:

- **Users:** improved search and interaction
- **Admins:** faster content management and better visibility
- **System:** richer event data through images and attachments

Overall, the system becomes more functional and practical while preserving the core booking workflow. These features improve usability without changing the core system design.